



编译原理  
第六章  
中间代码生成

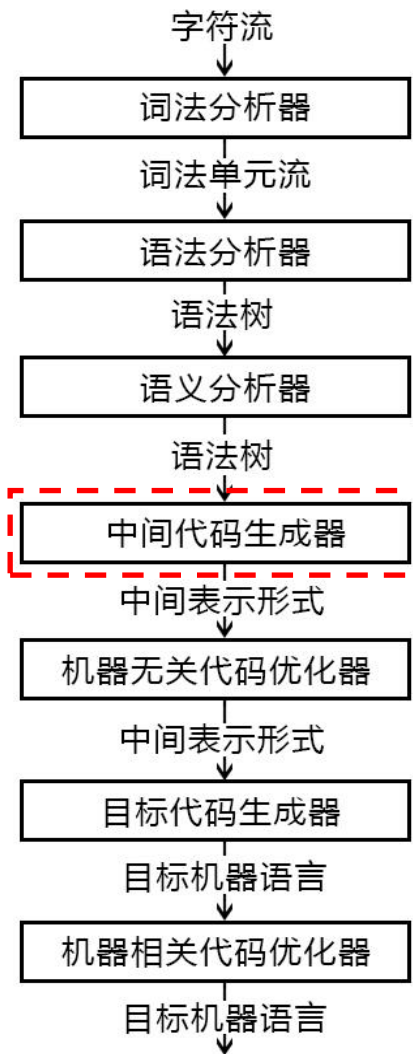
哈尔滨工业大学 陈鄞单丽莉



# 编译器的结构

## ➤ 中间代码的特点

- 简单规范
- 易于优化与转换
- 与机器无关
- 易于移植



# 常用的三地址指令

| 序号 | 指令类型         | 指令形式                                        |
|----|--------------|---------------------------------------------|
| 1  | 赋值指令         | $x = y \text{ op } z$<br>$x = \text{op } y$ |
| 2  | 复制指令         | $x = y$                                     |
| 3  | 条件跳转         | if $x \text{ relop } y$ goto $n$            |
| 4  | 非条件跳转        | goto $n$                                    |
| 5  | 参数传递         | param $x$                                   |
| 6  | 过程调用<br>函数调用 | call $p, n$<br>$y = \text{call } p, n$      |
| 7  | 过程返回         | return $x$                                  |
| 8  | 数组引用         | $x = y[i]$                                  |
| 9  | 数组赋值         | $x[i] = y$                                  |
| 10 | 地址及<br>指针操作  | $x = \& y$<br>$x = *y$<br>$*x = y$          |

三地址指令：一个运算符，三个地址；  
三地址：(最多)两个运算分量地址，  
一个结果分量地址。

地址可以具有如下形式之一

- 源程序中的名字 (name)
- 常量 (constant)
- 编译器生成的临时变量 (temporary)

- ◆ 指令中红色的部分表示指令的操作符
- ◆  $x[i]$ ，其中的 $i$ 是与首地址的偏移地址

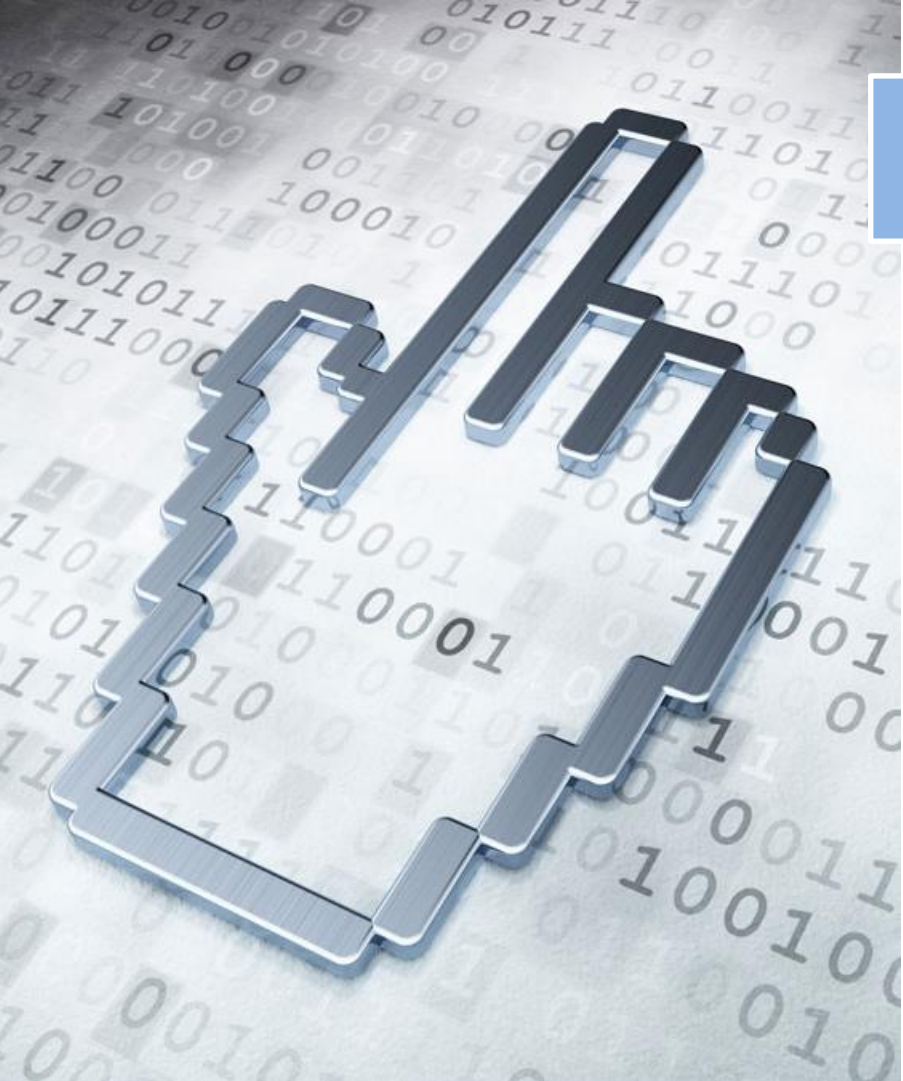
# 三地址指令的四元式表示

- $x = y \text{ op } z$       ( **op** ,  $y$  ,  $z$  ,  $x$  )
- $x = \text{op } y$       ( **op** ,  $y$  ,  $\_$  ,  $x$  )
- $x = y$       ( **=** ,  $y$  ,  $\_$  ,  $x$  )
- if  $x \text{ relop } y$  goto  $n$  ( **relop** ,  $x$  ,  $y$  ,  $n$  )
- **goto**  $n$       ( **goto** ,  $\_$  ,  $\_$  ,  $n$  )
- **param**  $x$       ( **param** ,  $\_$  ,  $\_$  ,  $x$  )
- $y = \text{call } p, n$       ( **call** ,  $p$  ,  $n$  ,  $y$  )
- return**  $x$       ( **return** ,  $\_$  ,  $\_$  ,  $x$  )
- $x = y[i]$       ( **=[]** ,  $y$  ,  $i$  ,  $x$  )
- $x[i] = y$       ( **[]=** ,  $y$  ,  $x$  ,  $i$  )
- $x = \&y$       ( **=\&** ,  $y$  ,  $\_$  ,  $x$  )
- $x = *y$       ( **=\*** ,  $y$  ,  $\_$  ,  $x$  )
- $*x = y$       ( **\*=** ,  $y$  ,  $\_$  ,  $x$  )



三地址指令序列唯一确定了运算完成的顺序

- 第一个分量是**操作符**
- 第二三个分量是**操作数**
- 第四个分量是**结果地址**



# 本章内容

## 6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 类型检查

6.4 控制语句的翻译

6.5 回填

6.6 switch语句的翻译

6.7 过程调用语句的翻译

## 6.1 声明语句的翻译

- 声明语句翻译的主要任务：收集标识符的类型等属性信息，并为每一个名字分配一个相对地址
- 类型的重要作用
  - 类型检查：验证程序运行时的行为是否遵守语言的类型规定。语义分析任务多数都与类型检查相关。
  - 辅助翻译：计算数组引用的地址、加入显式的类型转换、选择正确版本的算术运算符都要用到类型信息。

# 类型表达式 (*Type Expressions*)

➤ 基本类型是类型表达式

➤ *integer*

➤ *real*

➤ *char*

➤ *boolean*

➤ *type\_error* (出错类型)

➤ *void* (无类型)

# 类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，**类型名**也是类型表达式
- 将**类型构造符**(*type constructor*)作用于**类型表达式**可以构成新的类型表达式
  - **数组构造符***array*
    - 若*T*是类型表达式，则***array (I, T)***是类型表达式(*I*是一个整数)

| 类型                | 类型表达式                                            |
|-------------------|--------------------------------------------------|
| <i>int</i> [3]    | <i>array</i> (3, <i>int</i> )                    |
| <i>int</i> [2][3] | <i>array</i> (2, <i>array</i> (3, <i>int</i> ) ) |



# 类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，类型名也是类型表达式
- 将类型构造符(*type constructor*)作用于类型表达式可以构成新的类型表达式
  - 数组构造符 *array*
  - 指针构造符 *pointer*
    - 若  $T$  是类型表达式，则 *pointer* ( $T$ ) 是类型表达式，它表示一个指针类型

# 类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，类型名也是类型表达式
- 将类型构造符(*type constructor*)作用于类型表达式可以构成新的类型表达式
  - 数组构造符 *array*
  - 指针构造符 *pointer*
  - 笛卡尔乘积构造符  $\times$ 
    - 若  $T_1$  和  $T_2$  是类型表达式，则笛卡尔乘积  $T_1 \times T_2$  是类型表达式

# 类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，类型名也是类型表达式
- 将类型构造符(*type constructor*)作用于类型表达式可以构成新的类型表达式
  - 数组构造符 *array*
  - 指针构造符 *pointer*
  - 笛卡尔乘积构造符  $\times$
  - 函数构造符  $\rightarrow$ 
    - 若  $T_1$ 、 $T_2$ 、...、 $T_n$  和  $R$  是类型表达式，则  $T_1 \times T_2 \times \dots \times T_n \rightarrow R$  是类型表达式

# 类型表达式 (*Type Expressions*)

- 基本类型是类型表达式
- 可以为类型表达式命名，类型名也是类型表达式
- 将类型构造符(*type constructor*)作用于类型表达式可以构成新的类型表达式
  - 数组构造符 *array*
  - 指针构造符 *pointer*
  - 笛卡尔乘积构造符  $\times$
  - 函数构造符  $\rightarrow$
  - 记录构造符 *record*
    - 若有标识符  $N_1, N_2, \dots, N_n$  与类型表达式  $T_1, T_2, \dots, T_n$ ，则  $\text{record}((N_1 \times T_1) \times (N_2 \times T_2) \times \dots \times (N_n \times T_n))$  是一个类型表达式

# 例

➤ 设有C程序片段：

```
struct stype  
{ char[8] name;  
  int score;  
};  
stype[50] table;  
stype* p;
```

➤ 和*stype*绑定的类型表达式

➤ *record* ( (*name* × *array*(8, *char*)) × (*score* × *integer*) )

➤ 和*table*绑定的类型表达式

➤ *array* (50, *stype*)

➤ 和*p*绑定的类型表达式

➤ *pointer* (*stype*)

# 类型等价

- 许多类型检查都需要判断两个类型表达式是否等价：
- 结构等价：当且仅当下列条件之一成立时，称两个类型 $T_1$ 和 $T_2$ 是结构等价的：
  - $T_1$ 和 $T_2$ 是相同的基本类型；
  - $T_1$ 和 $T_2$ 是将同一类型构造符应用于结构等价的类型上形成的；
  - $T_1$ 是表示 $T_2$ 的类型名。
- 名字等价：两个类型表达式名字等价当且仅当上述前两条之一成立。 关注类型名称的一致性。

# 类型等价

例：有一段程序声明

```
typedef cell* link;
```

```
link next, last;
```

```
cell * p, q;
```

其中声明变量的类型表达式

| 变量 | 类型表达式 |
|----|-------|
|----|-------|

|      |      |
|------|------|
| next | link |
|------|------|

|      |      |
|------|------|
| last | link |
|------|------|

|   |               |
|---|---------------|
| p | pointer(cell) |
|---|---------------|

|   |               |
|---|---------------|
| q | pointer(cell) |
|---|---------------|

按名字等价判断：变量next和last类型相同，  
变量p, q类型相同。

按结构等价判断：所有4个变量类型都相同。

# 局部变量的存储分配

- 对于声明语句，语义分析的主要任务就是①收集标识符的类型等属性信息，②为每一个名字分配一个相对地址
- 从类型表达式可以知道该类型在运行时刻所需的存储单元数量称为类型的宽度(width)
- 在编译时刻，可以根据类型的宽度为每一个名字分配一个相对地址（相对于数据区域开始位置的偏移量）
- 名字的类型和相对地址信息保存在符号表记录中



# 变量声明语句的SDT

$enter( name, type, offset )$ : 在符号表中为名字  $name$  创建记录, 将  $name$  的类型设置为  $type$ , 相对地址设置为  $offset$

- ①  $P \rightarrow \{ offset = 0 \} D$
- ②  $D \rightarrow T id; \{ \underline{enter( id.lexeme, T.type, offset )};$   
 $\quad \quad \quad \underline{offset = offset + T.width; } D$
- ③  $D \rightarrow \varepsilon$
- ④  $T \rightarrow B \quad \{ \underline{t = B.type; w = B.width; } \}$   
 $\quad \quad \quad C \quad \{ T.type = C.type; T.width = C.width; \}$
- ⑤  $T \rightarrow \uparrow T_1 \{ T.type = pointer( T_1.type ); T.width = 4; \}$
- ⑥  $B \rightarrow int \{ B.type = int; B.width = 4; \}$
- ⑦  $B \rightarrow real \{ B.type = real; B.width = 8; \}$
- ⑧  $C \rightarrow \varepsilon \quad \{ C.type = \underline{t}; C.width = \underline{w}; \}$
- ⑨  $C \rightarrow [num] C_1 \{ C.type = array( num.val, C_1.type );$   
 $\quad \quad \quad \underline{C.width = num.val * C_1.width; } \}$

| 符号  | 综合属性          |
|-----|---------------|
| $B$ | $type, width$ |
| $C$ | $type, width$ |
| $T$ | $type, width$ |

| 变量       | 作用                                                               |
|----------|------------------------------------------------------------------|
| $offset$ | 下一个可用的相对地址                                                       |
| $t, w$   | 将类型和宽度信息从语法分析树中的 $B$ 结点传递到对应于产生式 $C \rightarrow \varepsilon$ 的结点 |

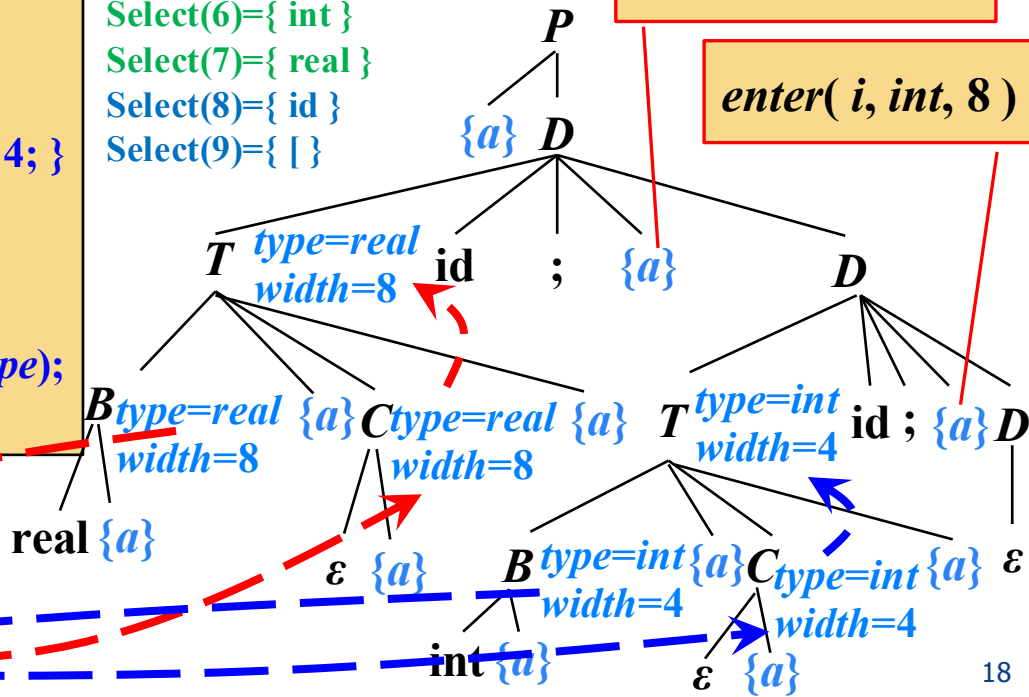
# 例: “*real x; int i;*”的语法制导翻译



Select(1)={int,real, ↑,\$}  
 Select(2)={int,real, ↑}  
 Select(3)={\$}  
 Select(4)={int,real}  
 Select(5)={ ↑ }  
 Select(6)={ int }  
 Select(7)={ real }  
 Select(8)={ id }  
 Select(9)={ [ ] }

*enter( x, real, 0 )*

*enter( i, int, 8 )*

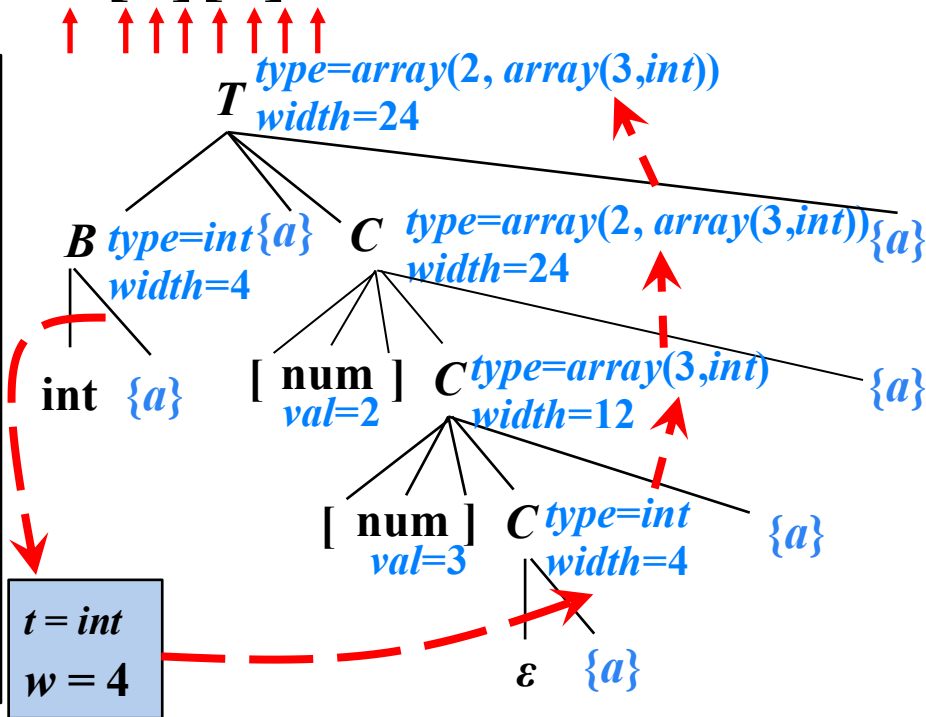


*offset = 12*

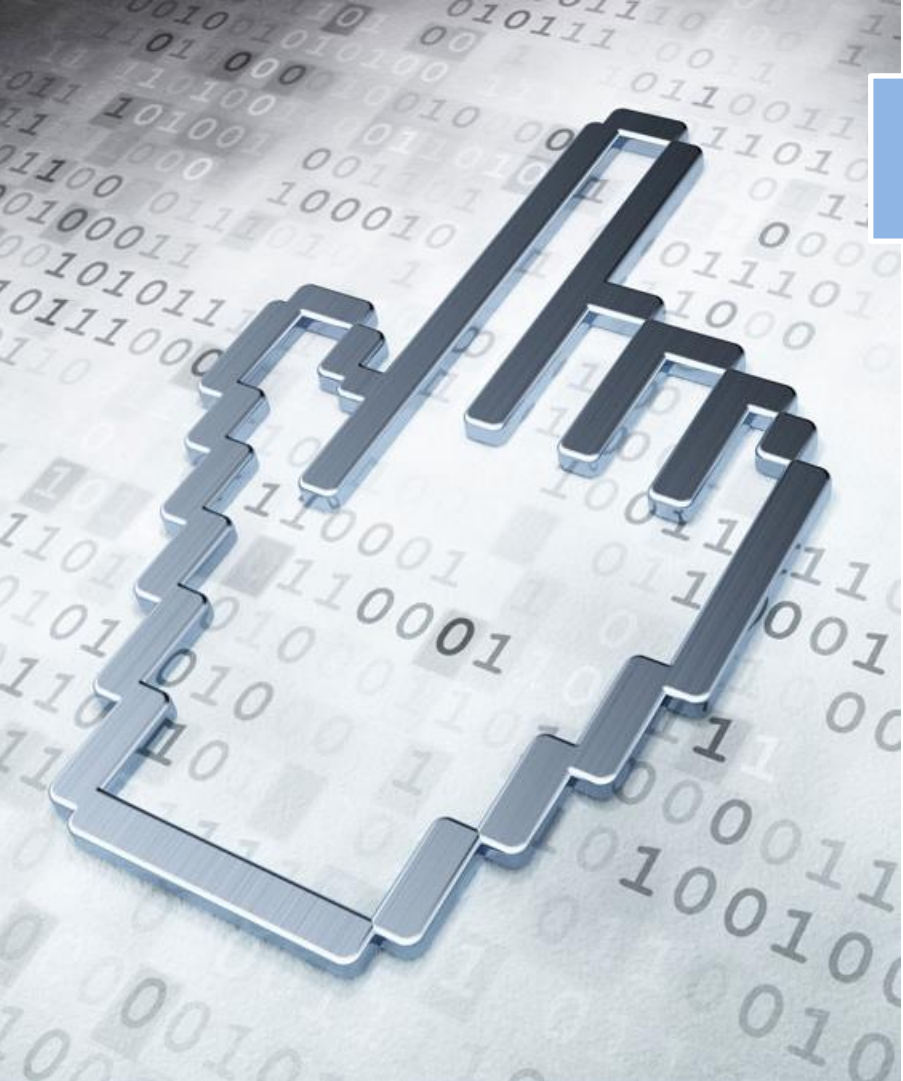
*t = int*  
*w = 4*

# 例：数组类型表达式“*int*[2][3]”的语法制导翻译

- ①  $P \rightarrow \{ \text{offset} = 0 \} D$
- ②  $D \rightarrow T \text{ id}; \{ \text{enter}(\text{id.lexeme}, T.\text{type}, \text{offset}); \text{offset} = \text{offset} + T.\text{width}; \} D$
- ③  $D \rightarrow \varepsilon$
- ④  $T \rightarrow B \quad \{ \underline{t = B.\text{type}}; \underline{w = B.\text{width}}; \}$   
 $\quad C \quad \{ T.\text{type} = C.\text{type}; T.\text{width} = C.\text{width}; \}$
- ⑤  $T \rightarrow \uparrow T_1 \{ T.\text{type} = \text{pointer}(T_1.\text{type}); T.\text{width} = 4; \}$
- ⑥  $B \rightarrow \text{int} \{ B.\text{type} = \text{int}; B.\text{width} = 4; \}$
- ⑦  $B \rightarrow \text{real} \{ B.\text{type} = \text{real}; B.\text{width} = 8; \}$
- ⑧  $C \rightarrow \varepsilon \quad \{ \underline{C.\text{type} = t}; \underline{C.\text{width} = w}; \}$
- ⑨  $C \rightarrow [\text{num}] C_1 \{ C.\text{type} = \text{array}(\text{num.val}, C_1.\text{type}); C.\text{width} = \text{num.val} * C_1.\text{width}; \}$



数组：只计算和保存整个数组的起始相对地址。



# 本章内容

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 类型检查

6.4 控制语句的翻译

6.5 回填

6.6 switch语句的翻译

6.7 过程调用语句的翻译

## 6.2 赋值语句的翻译

- 6.2.1 简单赋值语句的翻译
- 6.2.2 数组引用的翻译

## 6.2.1 简单赋值语句的翻译

➤ 赋值语句的基本文法    ➤ 赋值语句翻译的主要任务

①  $S \rightarrow \text{id} = E;$

②  $E \rightarrow E_1 + E_2$

③  $E \rightarrow E_1 * E_2$

④  $E \rightarrow -E_1$

⑤  $E \rightarrow (E_1)$

⑥  $E \rightarrow \text{id}$

➤ 生成对表达式求值的中间代码

➤ 例  $x = (a + b) * c;$

➤ 三地址码

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$x = t_2$$

# 赋值语句的SDT

*lookup(id.lexeme)*: 查询符号  
表返回 *id* 对应的地址

$S \rightarrow id = E;$  { *p* = *lookup(id.lexeme)*; if *p* == nil then error ;

*S.code* = *E.code* ||

*gen*(*p* '=' *E.addr*); }

*gen*(*code*): 生成三地址指令*code*

$E \rightarrow E_1 + E_2$  { *E.addr* = *newtemp*();

*E.code* = *E<sub>1</sub>.code* || *E<sub>2</sub>.code* ||

*gen*(*E.addr* '=' *E<sub>1</sub>.addr* '+' *E<sub>2</sub>.addr*); }

*newtemp*( ): 生成一个新的临时变量*t*,  
并返回*t*的地址

$E \rightarrow E_1 * E_2$  { *E.addr* = *newtemp*();

*E.code* = *E<sub>1</sub>.code* || *E<sub>2</sub>.code* ||

*gen*(*E.addr* '=' *E<sub>1</sub>.addr* '\*' *E<sub>2</sub>.addr*); }

$E \rightarrow -E_1$  { *E.addr* = *newtemp*();

*E.code* = *E<sub>1</sub>.code* ||

*gen*(*E.addr* '=' 'uminus' *E<sub>1</sub>.addr*); }

$E \rightarrow (E_1)$  { *E.addr* = *E<sub>1</sub>.addr*;

*E.code* = *E<sub>1</sub>.code*; }

$E \rightarrow id$  { *E.addr* = *lookup*(*id.lexeme*); if *E.addr* == nil then error ;

*E.code* = ''; }

| 符号       | 综合属性                       |
|----------|----------------------------|
| <i>S</i> | <i>code</i>                |
| <i>E</i> | <i>code</i><br><i>addr</i> |

# 增量翻译 (*Incremental Translation*)

$S \rightarrow id = E;$  {  $p = lookup(id.lexeme);$  if  $p == nil$  then error ;  
                   $S.code = E.code$  ||  
                   $gen(p '=' E.addr);$  }

$E \rightarrow E_1 + E_2$  {  $E.addr = newtemp();$   
                   $E.code = E_1.code$  ||  $E_2.code$  ||  
                   $gen(E.addr '=' E_1.addr '+' E_2.addr);$  }

$E \rightarrow E_1 * E_2$  {  $E.addr = newtemp();$   
                   $E.code = E_1.code$  ||  $E_2.code$  ||  
                   $gen(E.addr '=' E_1.addr '*' E_2.addr);$  }

$E \rightarrow -E_1$  {  $E.addr = newtemp();$   
                   $E.code = E_1.code$  ||  
                   $gen(E.addr '=' 'uminus' E_1.addr);$  }

$E \rightarrow (E_1)$  {  $E.addr = E_1.addr;$   
                   $E.code = E_1.code;$  }

$E \rightarrow id$  {  $E.addr = lookup(id.lexeme);$  if  $E.addr == nil$  then error ;  
                   $E.code = '';$  }

不再使用`code`属性，`gen()`负责构造出一个新的三地址指令，再将它添加到至今为止已生成的指令序列之后

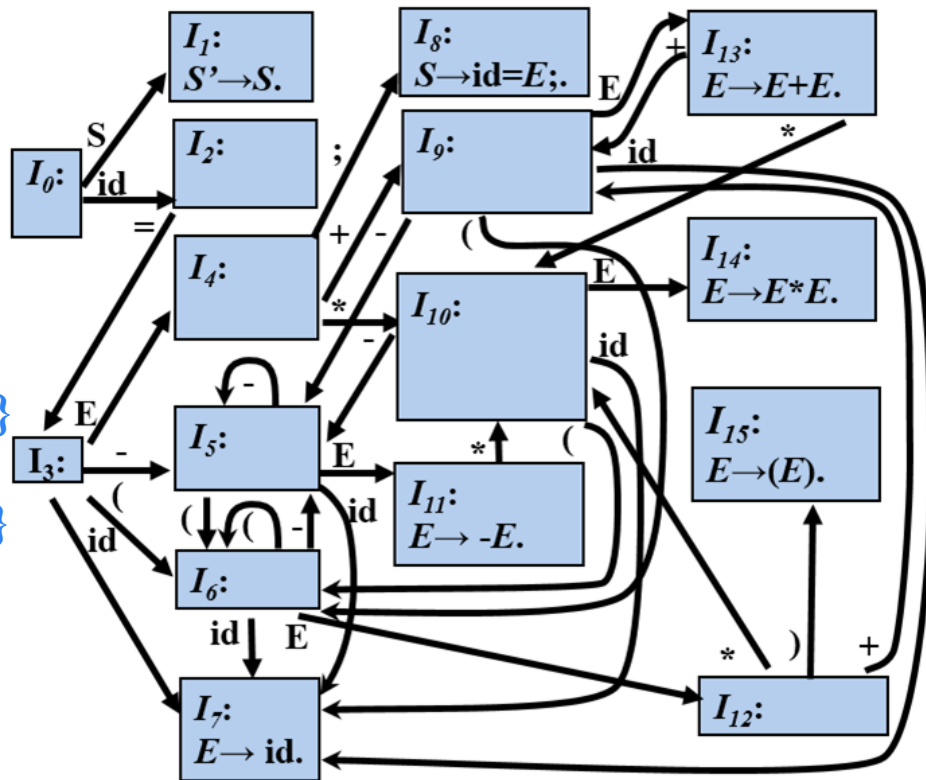
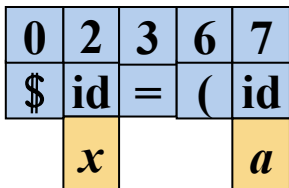
◆ 综合属性`code`的值总是将产生式右部符号对应的三地址码按生成顺序连接以后，在后面追加新的三地址指令



 **例**

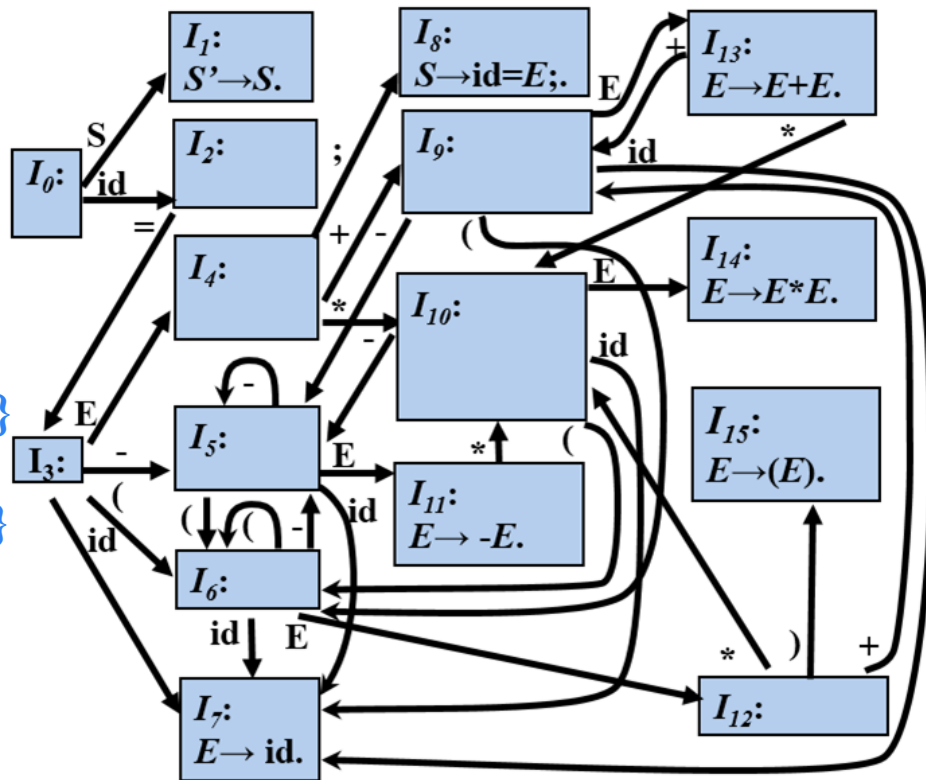
- ①  $S \rightarrow \text{id} = E$ ; {  $p = \text{lookup}(\text{id.lexeme})$ ;  
if  $p == \text{nil}$  then error ;  
gen(  $p = E.\text{addr}$  ); }
- ②  $E \rightarrow E_1 + E_2$  {  $E.\text{addr} = \text{newtemp}()$ ;  
gen( $E.\text{addr} = E_1.\text{addr} + E_2.\text{addr}$ ); }
- ③  $E \rightarrow E_1 * E_2$  {  $E.\text{addr} = \text{newtemp}()$ ;  
gen( $E.\text{addr} = E_1.\text{addr} * E_2.\text{addr}$ ); }
- ④  $E \rightarrow -E_1$  {  $E.\text{addr} = \text{newtemp}()$ ;  
gen( $E.\text{addr} = \text{'uminus'} E_1.\text{addr}$ ); }
- ⑤  $E \rightarrow (E_1)$  {  $E.\text{addr} = E_1.\text{addr}$  ; }
- ⑥  $E \rightarrow \text{id}$  {  $E.\text{addr} = \text{lookup}(\text{id.lexeme})$ ;  
if  $E.\text{addr} == \text{nil}$  then error ; }

例:  $x = (a + b) * c;$



# 例

- ①  $S \rightarrow id = E; \{ p = lookup(id.lexeme);$   
*if*  $p == nil$  *then error* ;  
 $gen(p '=' E.addr); \}$
- ②  $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '+' E_2.addr); \}$
- ③  $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '*' E_2.addr); \}$
- ④  $E \rightarrow -E_1 \{ E.addr = newtemp();$   
 $gen(E.addr '=' 'uminus' E_1.addr); \}$
- ⑤  $E \rightarrow (E_1) \{ E.addr = E_1.addr; \}$
- ⑥  $E \rightarrow id \{ E.addr = lookup(id.lexeme);$   
*if*  $E.addr == nil$  *then error* ;  $\}$



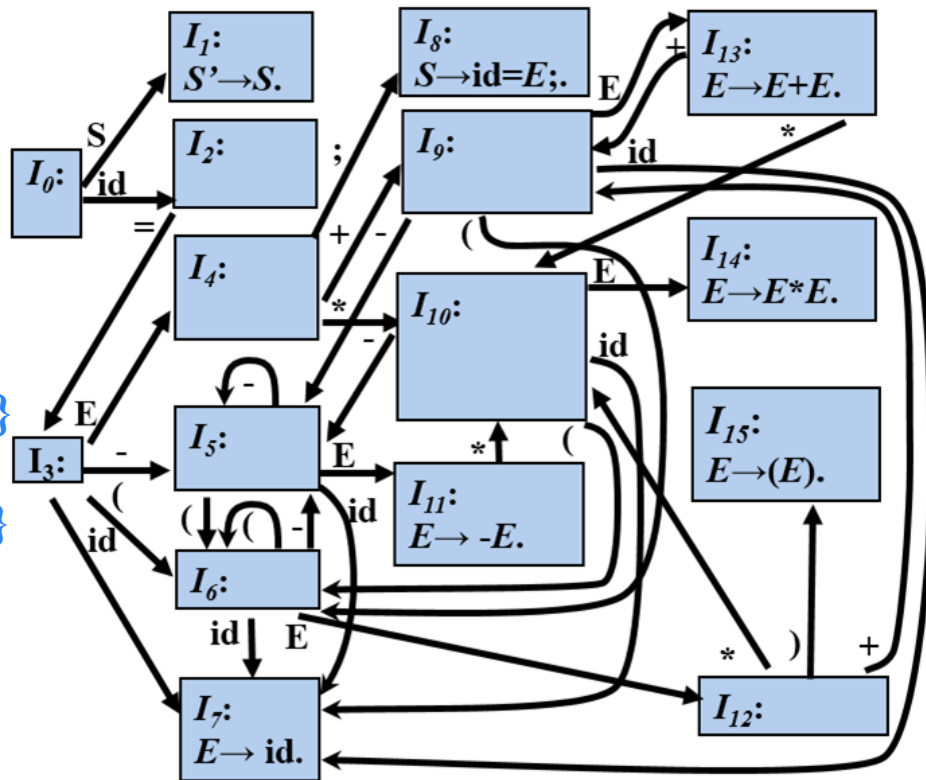
例:  $x = (a + b) * c;$



|    |    |   |   |    |   |    |
|----|----|---|---|----|---|----|
| 0  | 2  | 3 | 6 | 12 | 9 | 7  |
| \$ | id | = | ( | E  | + | id |
|    | x  |   |   | a  |   | b  |

# 例

- ①  $S \rightarrow id = E; \{ p = lookup(id.lexeme);$   
*if*  $p == nil$  *then error* ;  
 $gen(p '=' E.addr); \}$
- ②  $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '+' E_2.addr); \}$
- ③  $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '*' E_2.addr); \}$
- ④  $E \rightarrow -E_1 \{ E.addr = newtemp();$   
 $gen(E.addr '=' 'uminus' E_1.addr); \}$
- ⑤  $E \rightarrow (E_1) \{ E.addr = E_1.addr; \}$
- ⑥  $E \rightarrow id \{ E.addr = lookup(id.lexeme);$   
*if*  $E.addr == nil$  *then error* ;  $\}$



$$t_1 = a + b$$

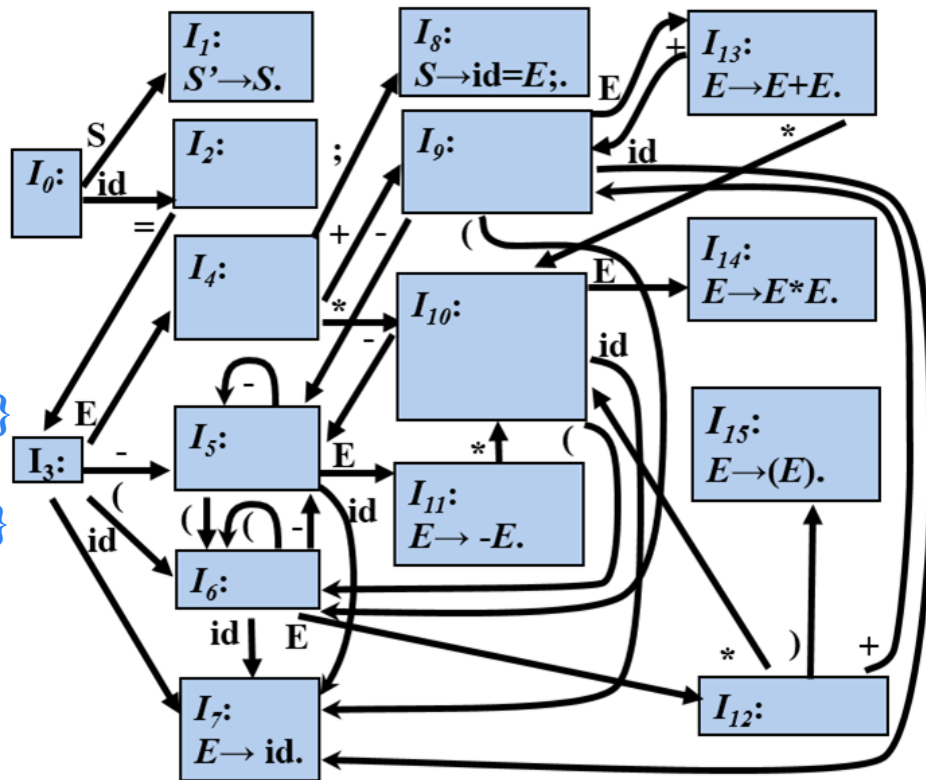
例:  $x = (a + b) * c;$



|    |    |   |   |    |   |    |
|----|----|---|---|----|---|----|
| 0  | 2  | 3 | 6 | 12 | 9 | 13 |
| \$ | id | = | ( | E  | + | E  |
|    | x  |   |   | a  |   | b  |

# 例

- ①  $S \rightarrow id = E; \{ p = lookup(id.lexeme);$   
*if*  $p == nil$  *then error* ;  
 $gen(p '=' E.addr); \}$
- ②  $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '+' E_2.addr); \}$
- ③  $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '*' E_2.addr); \}$
- ④  $E \rightarrow -E_1 \{ E.addr = newtemp();$   
 $gen(E.addr '=' 'uminus' E_1.addr); \}$
- ⑤  $E \rightarrow (E_1) \{ E.addr = E_1.addr; \}$
- ⑥  $E \rightarrow id \{ E.addr = lookup(id.lexeme);$   
*if*  $E.addr == nil$  *then error* ;  $\}$



$$t_1 = a + b$$

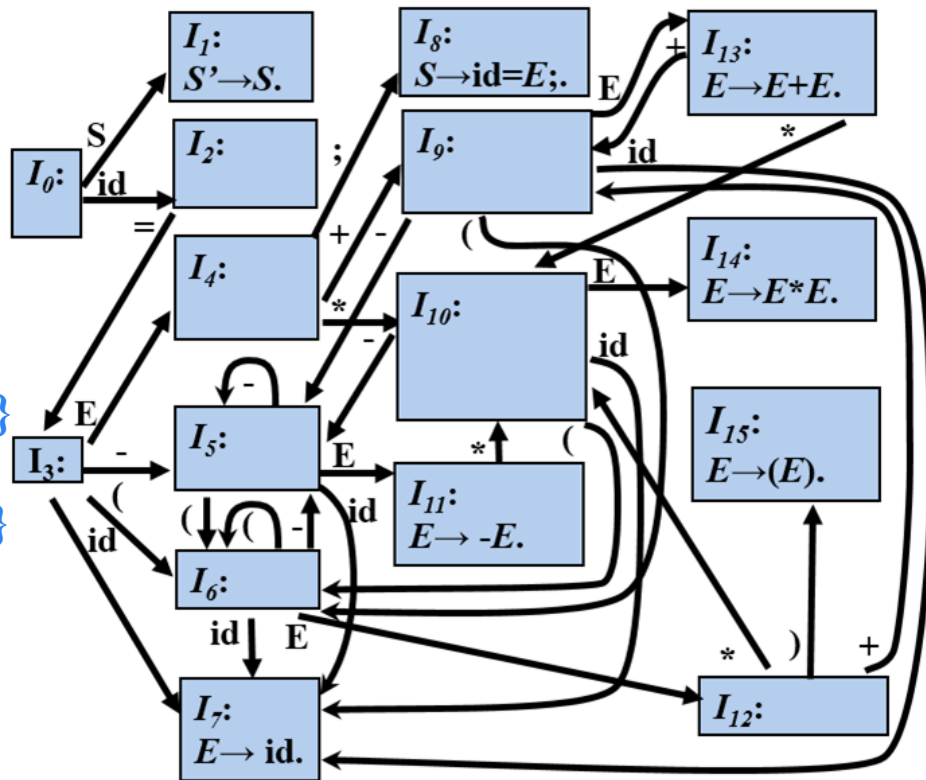
例:  $x = (a + b) * c;$



|    |    |   |   |                |    |
|----|----|---|---|----------------|----|
| 0  | 2  | 3 | 6 | 12             | 15 |
| \$ | id | = | ( | E              | )  |
|    | x  |   |   | t <sub>1</sub> |    |

# 例

- ①  $S \rightarrow id = E; \{ p = lookup(id.lexeme);$   
*if*  $p == nil$  *then error* ;  
 $gen(p '=' E.addr); \}$
- ②  $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '+' E_2.addr); \}$
- ③  $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '*' E_2.addr); \}$
- ④  $E \rightarrow -E_1 \{ E.addr = newtemp();$   
 $gen(E.addr '=' 'uminus' E_1.addr); \}$
- ⑤  $E \rightarrow (E_1) \{ E.addr = E_1.addr; \}$
- ⑥  $E \rightarrow id \{ E.addr = lookup(id.lexeme);$   
*if*  $E.addr == nil$  *then error* ;  $\}$



$$t_1 = a + b$$

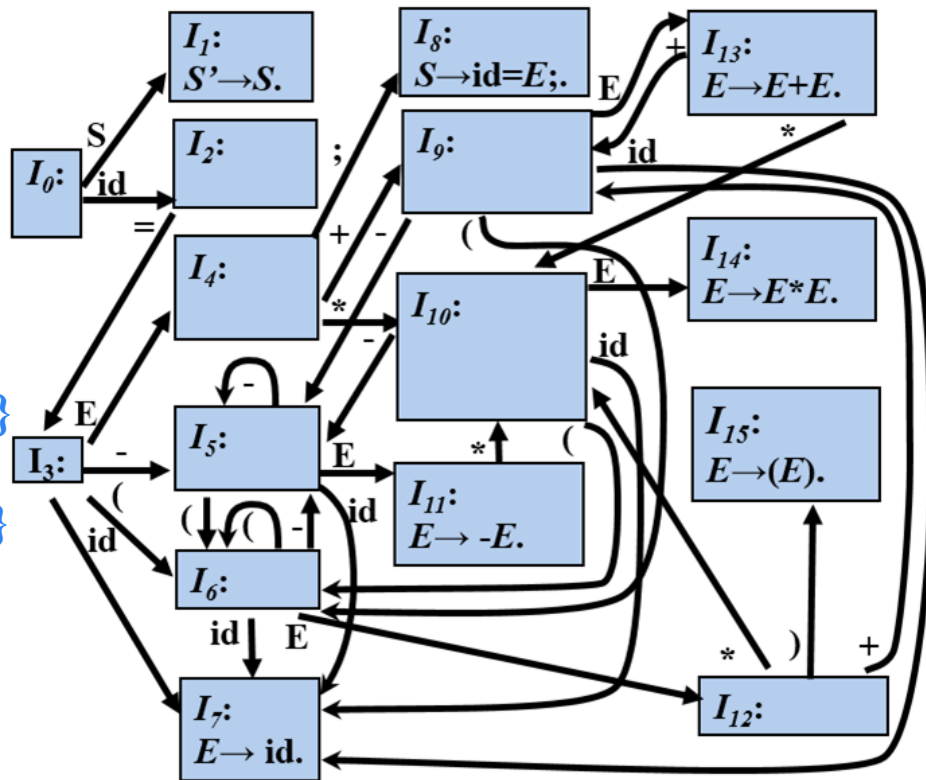
例:  $x = (a + b) * c;$



|    |    |   |                |    |    |
|----|----|---|----------------|----|----|
| 0  | 2  | 3 | 4              | 10 | 7  |
| \$ | id | = | E              | *  | id |
|    | x  |   | t <sub>1</sub> |    | c  |

# 例

- ①  $S \rightarrow id = E; \{ p = lookup(id.lexeme);$   
*if*  $p == nil$  *then error* ;  
 $gen(p '=' E.addr); \}$
- ②  $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '+' E_2.addr); \}$
- ③  $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '*' E_2.addr); \}$
- ④  $E \rightarrow -E_1 \{ E.addr = newtemp();$   
 $gen(E.addr '=' 'uminus' E_1.addr); \}$
- ⑤  $E \rightarrow (E_1) \{ E.addr = E_1.addr; \}$
- ⑥  $E \rightarrow id \{ E.addr = lookup(id.lexeme);$   
*if*  $E.addr == nil$  *then error* ;  $\}$



例:  $x = (a + b) * c;$



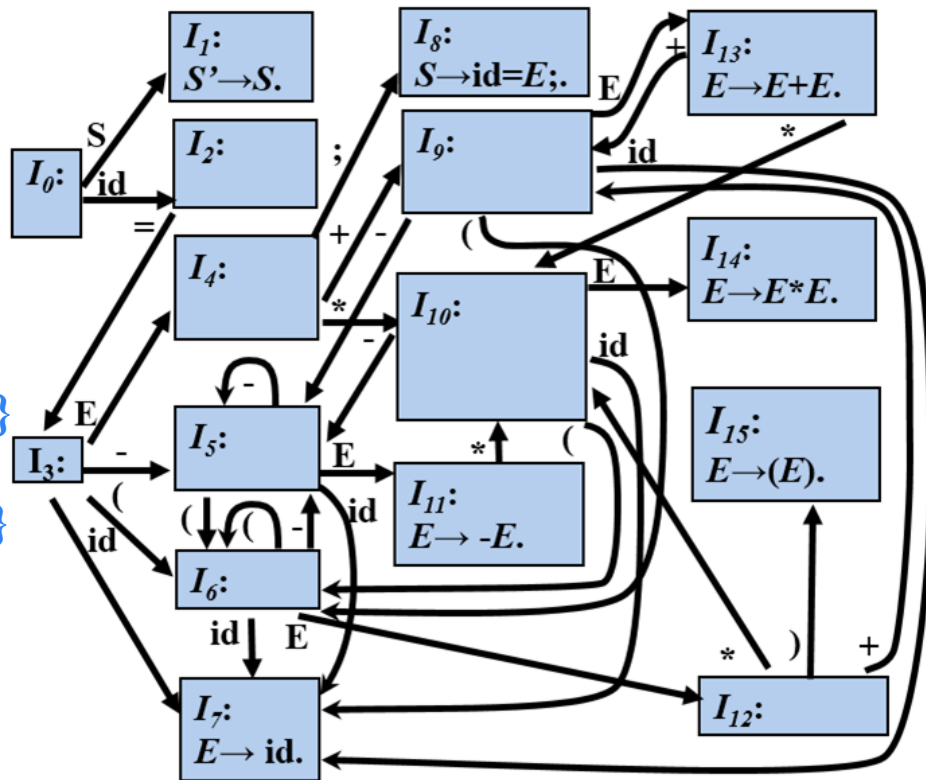
| 0  | 2  | 3 | 4              | 10 | 14 |
|----|----|---|----------------|----|----|
| \$ | id | = | E              | *  | E  |
|    | x  |   | t <sub>1</sub> |    | c  |

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

# 例

- ①  $S \rightarrow id = E; \{ p = lookup(id.lexeme);$   
*if*  $p == nil$  *then error* ;  
 $gen(p '=' E.addr); \}$
- ②  $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '+' E_2.addr); \}$
- ③  $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '*' E_2.addr); \}$
- ④  $E \rightarrow -E_1 \{ E.addr = newtemp();$   
 $gen(E.addr '=' 'uminus' E_1.addr); \}$
- ⑤  $E \rightarrow (E_1) \{ E.addr = E_1.addr ; \}$
- ⑥  $E \rightarrow id \{ E.addr = lookup(id.lexeme);$   
*if*  $E.addr == nil$  *then error* ;  $\}$



例:  $x = (a + b) * c ;$



|    |    |   |                |   |
|----|----|---|----------------|---|
| 0  | 2  | 3 | 4              | 8 |
| \$ | id | = | E              | ; |
|    | x  |   | t <sub>2</sub> |   |

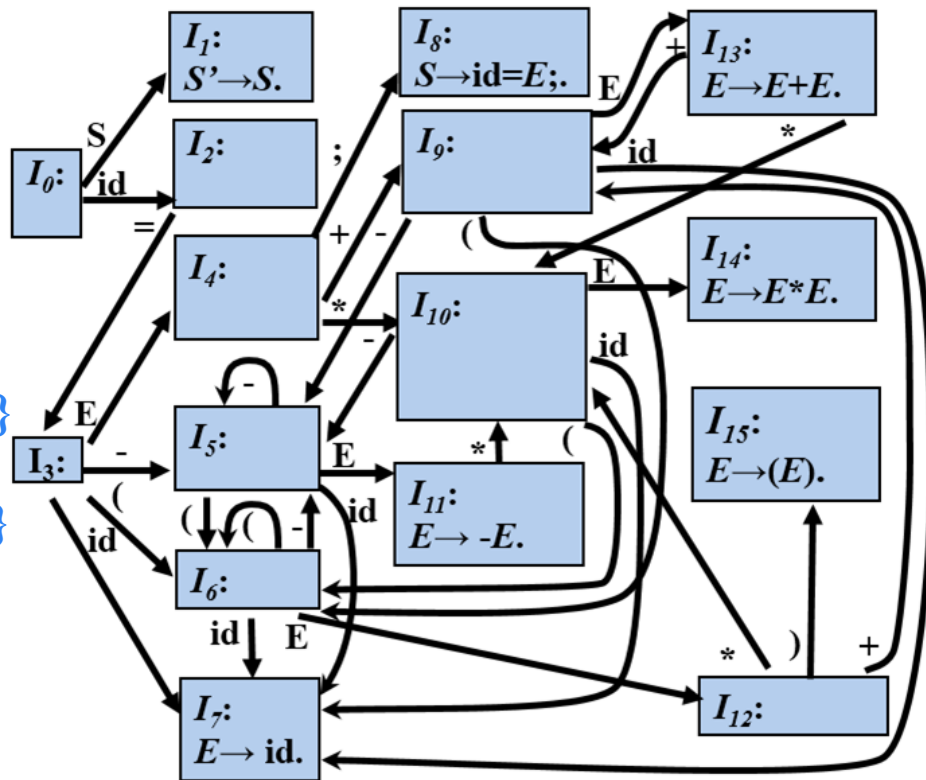
$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$x = t_2$$

# 例

- ①  $S \rightarrow id = E; \{ p = lookup(id.lexeme);$   
*if*  $p == nil$  *then error* ;  
 $gen(p '=' E.addr); \}$
- ②  $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '+' E_2.addr); \}$
- ③  $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$   
 $gen(E.addr '=' E_1.addr '*' E_2.addr); \}$
- ④  $E \rightarrow -E_1 \{ E.addr = newtemp();$   
 $gen(E.addr '=' 'uminus' E_1.addr); \}$
- ⑤  $E \rightarrow (E_1) \{ E.addr = E_1.addr; \}$
- ⑥  $E \rightarrow id \{ E.addr = lookup(id.lexeme);$   
*if*  $E.addr == nil$  *then error* ;  $\}$



例:  $x = (a + b) * c;$



|    |   |
|----|---|
| 0  | 1 |
| \$ | S |

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$x = t_2$$



## 6.2.2 数组引用的翻译

### ➤ 赋值语句的基本文法

$$S \rightarrow \text{id} = E; \mid L = E;$$
$$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id} \mid L$$
$$L \rightarrow \text{id} [E] \mid L_1 [E]$$

将数组引用翻译成三地址码时要解决的主要问题是确定数组元素的存放地址，也就是数组元素的寻址

# 例

➤  $\text{int } a[3][5][8];$

$\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{int})))$ ,

一个整型变量占用4个字节,

则  $\text{addr}(a[2][3][4]) = \text{base} + 2 * w_1 + 3 * w_2 + 4 * w_3$

$$= \text{base} + 2 * 160 + 3 * 32 + 4 * 4$$

$a[2]$  类型  $\text{array}(5, \text{array}(8, \text{int}))$  的宽度

$a[2][3]$  的类型  $\text{array}(8, \text{int})$  宽度

$a[2][3][4]$  的类型  $\text{int}$  宽度

## 数组元素寻址 (*Addressing Array Elements*)

➤ 一般情况,  $k$  维数组

➤ 数组元素  $a[i_1] [i_2] \dots [i_k]$  的相对地址是:

$$\text{base} + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k$$

偏移地址

$\text{array}(n_1, \text{array}(n_2, \text{array}(n_3, \dots$   
 $\dots \text{array}(n_k, \text{type}) \dots))$

$w_1 \rightarrow a[i_1]$  的宽度

$w_2 \rightarrow a[i_1] [i_2]$  的宽度

...

$w_k \rightarrow a[i_1] [i_2] \dots [i_k]$  的宽度

# 带有数组引用的赋值语句的翻译

## ➤ 例1

假设  $type(a)=array(n, int)$ ,

### ➤ 源程序片段

➤  $c = a[i];$

$$addr(a[i]) = base + i * 4$$

### ➤ 三地址码

$$t_1 = i * 4$$

$$t_2 = a[t_1]$$

$$c = t_2$$

# 带有数组引用的赋值语句的翻译

## ➤ 例2

假设  $type(a) = array(3, array(5, int))$ ,

### ➤ 源程序片段

➤  $c = a[i_1][i_2];$   $addr(a[i_1][i_2]) = base + \underbrace{i_1 * 20}_{t_1} + \underbrace{i_2 * 4}_{t_2}$

### ➤ 三地址码

$$t_1 = i_1 * 20$$

$$t_2 = i_2 * 4$$

$$t_3 = t_1 + t_2$$

$$t_4 = a[t_3]$$

$$c = t_4$$

# 数组元素寻址的SDT

## ➤ 赋值语句的基本文法

$S \rightarrow \text{id} = E; \mid L = E;$

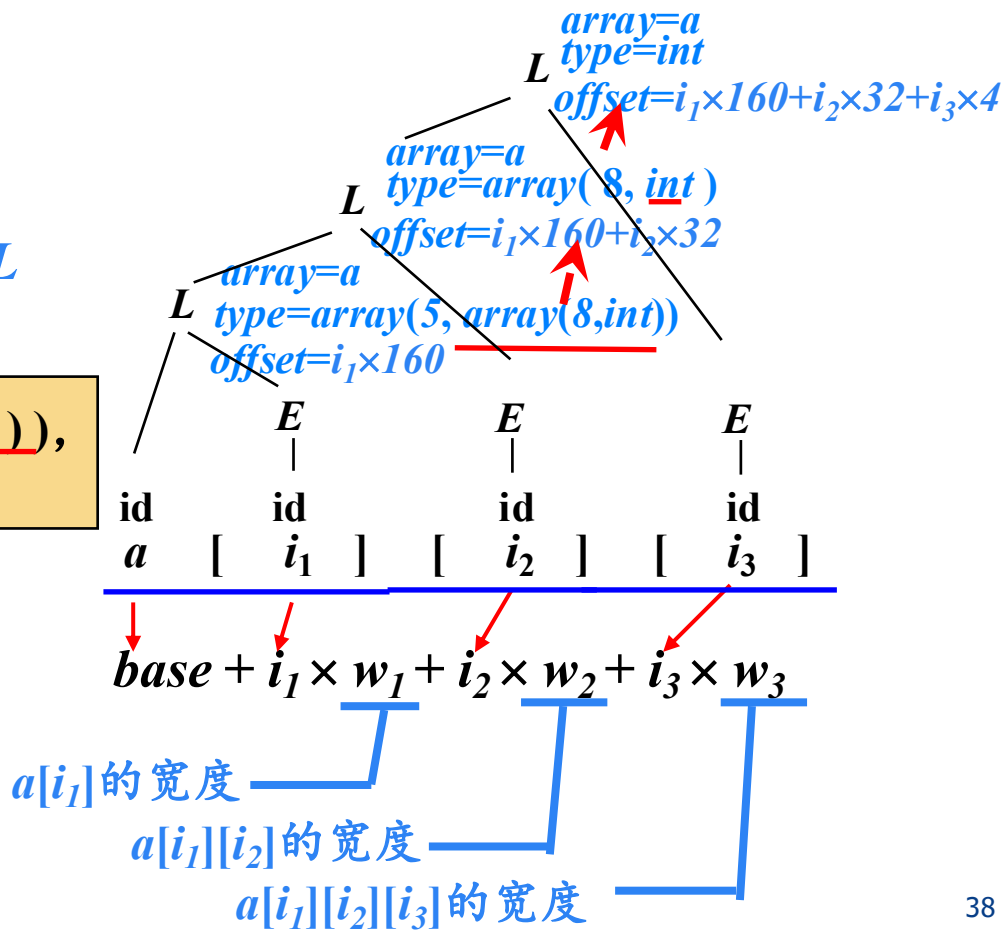
$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id} \mid L$

$L \rightarrow \text{id} [E] \mid L_1 [E]$

假设  $\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{int})))$ ,  
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

### ➤ $L$ 的综合属性

- $L.\text{type}$ :  $L$  生成的数组元素的类型
- $L.\text{offset}$ : 指示一个临时变量, 该临时变量用于累加公式中的  $i_j \times w_j$  项, 从而计算数组元素的偏移量
- $L.\text{array}$ : 数组名在符号表的入口地址



# 数组元素寻址的SDT

假设  $type(a) = array(3, \underline{array(5, array(8, int))})$ ,  
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

$$addr(a[i_1][i_2][i_3]) = \underbrace{base}_{t_1} + \underbrace{i_1 \times w_1}_{t_2} + \underbrace{i_2 \times w_2}_{t_3} + \underbrace{i_3 \times w_3}_{t_4}$$

$S \rightarrow id = E;$

$| L = E; \{ gen( L.array.base \text{ '[' } L.offset \text{ ']' } '=' E.addr ); \}$

$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid id$

$| L \{ E.addr = newtemp(); gen( E.addr '=' L.array.base \text{ '[' } L.offset \text{ ']' } ); \}$

$L \rightarrow id \ [E] \{ L.array = lookup(id.lexeme); if L.array == nil then error;$

$L.type = \underline{L.array.type.elem};$  //获得子数组类型

$\underline{L.offset = newtemp();}$

$\underline{gen( L.offset '=' E.addr '*' L.type.width );}$

$| L_1[E] \{ L.array = L_1.array;$

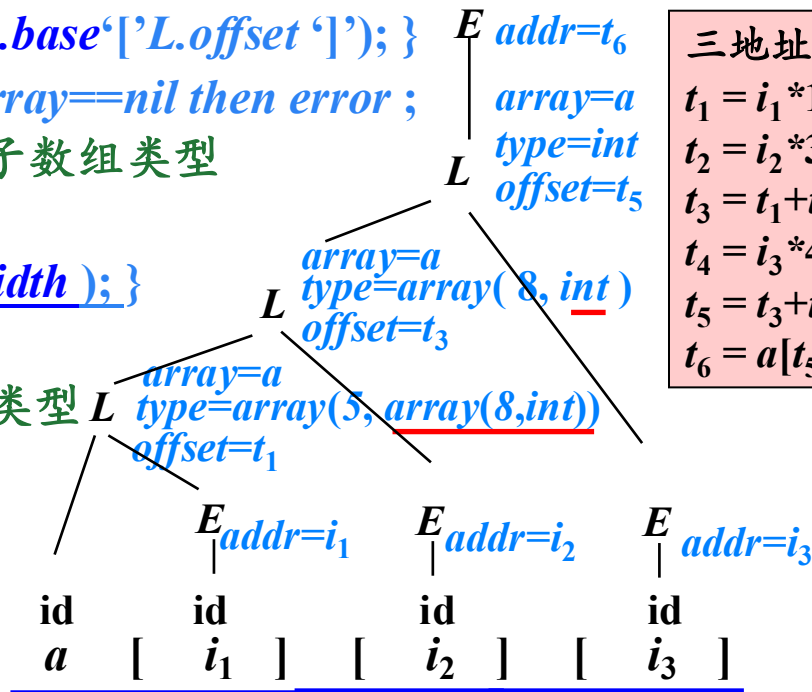
$L.type = L_1.type.elem;$  //获得子数组类型

$\underline{t = newtemp();}$

$\underline{gen( t '=' E.addr '*' L.type.width );}$

$\underline{L.offset = newtemp();}$

$\underline{gen( L.offset '=' L_1.offset '+' t );}$

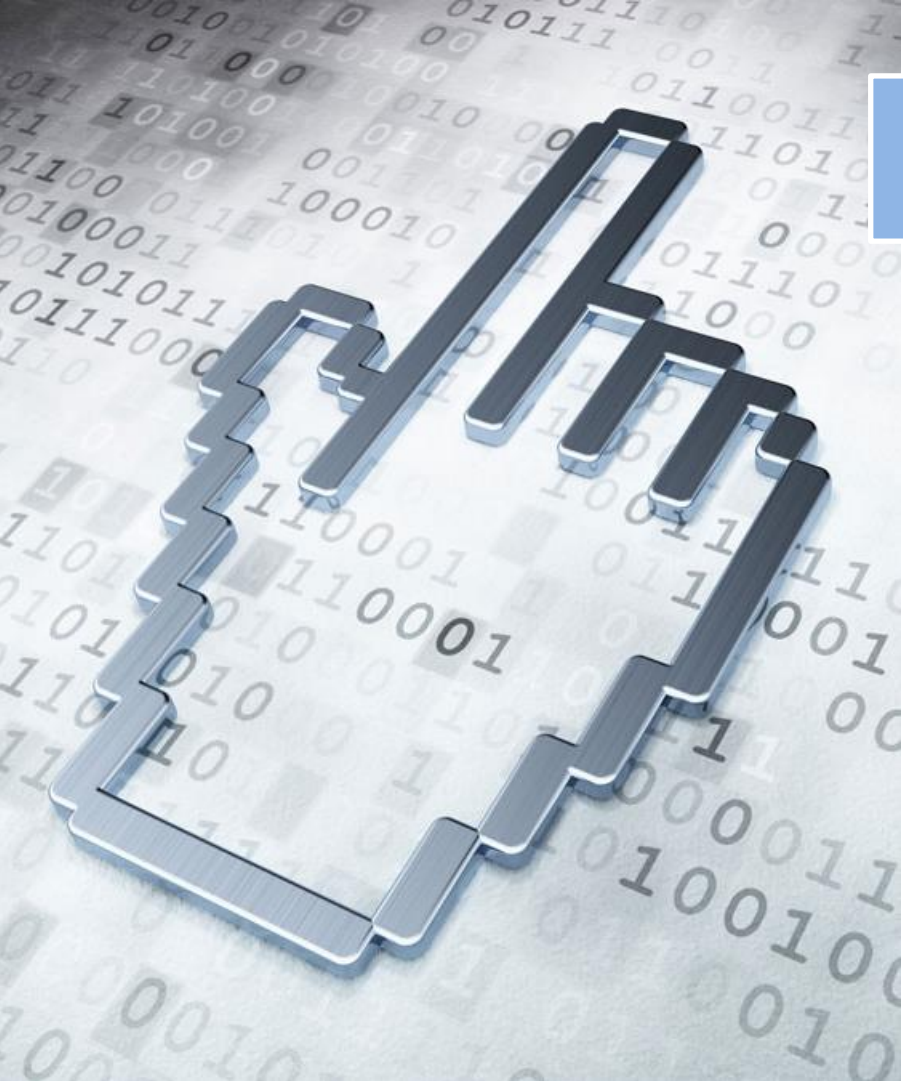


## 练习1

- 使用讲义中的翻译方案翻译下面赋值语句生成三地址码序列。假设 $a$ 和 $b$ 的类型表达式都是 `array(3, array(5, real))`, `real`类型占8个存储单元。

$$x = a[i][j] + b[i][j]$$





# 本章内容

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 类型检查

6.4 控制语句的翻译

6.5 回填

6.6 switch语句的翻译

6.7 过程调用语句的翻译

## 6.3 类型检查\*

- 为每个语法成分赋予一个类型表达式
  - 名字的类型表达式保存在符号表中
  - 其他语法成分(如表达式 $E$ 或语句 $S$ )的类型表达式则作为属性保存在语义栈中。
- 类型检查的任务就是确定这些类型表达式是否符合一定的规则，这些规则的集合通常称为源程序的类型系统。
- 类型检查具有发现程序错误的能力。

## 6.3.1 类型检查的规则

### ➤ 类型综合 (Type Synthesis)

- 从子表达式的类型通过规则推导出整个表达式的类型 (自底向上)
- 要求名字在引用之前必须先进行声明
- if  $f$  的类型为  $s \rightarrow t$  and  $x$  的类型为  $s$  then 表达式  $f(x)$  的类型为  $t$

```
double a = 5;  
double b = 3.14;  
int c = a + b;
```

类型综合

$a+b$  类型是double,  
可能需要类型转换

## 6.3.1 类型检查的规则

### ➤ 类型综合 (Type Synthesis)

### ➤ 类型推断 (Type Inference)

➤ 类型推断是根据上下文信息，自动推导出变量或表达式的类型，无需显式声明。编译器通过分析代码的约束关系（如变量赋值、函数参数传递）来推断类型。

➤ 经常被用于从函数体推断函数类型

➤ if  $f(x)$  是一个表达式 then 对某两个类型变量  $\alpha$  和  $\beta$ ,  $f$  具有类型  $\alpha \rightarrow \beta$  and  $x$  具有类型  $\alpha$

```
auto add( int t, double u) {  
    return t + u;  
}
```

类型推断

add:  $\text{int} \times \text{double} \rightarrow \text{double}$

## 6.3.2 类型转换

- 不同类型机内表示不同，且机器运算指令不同
- 编译器需要将运算分量转换为相同类型

```
int a = 5, b = 3.14;
```

```
float c = 2;
```

```
int z = a * b + c;
```

➤ 中间代码

```
t1 = a int* b
```

```
t2 = (float) t1
```

```
t3 = c float+ t2
```

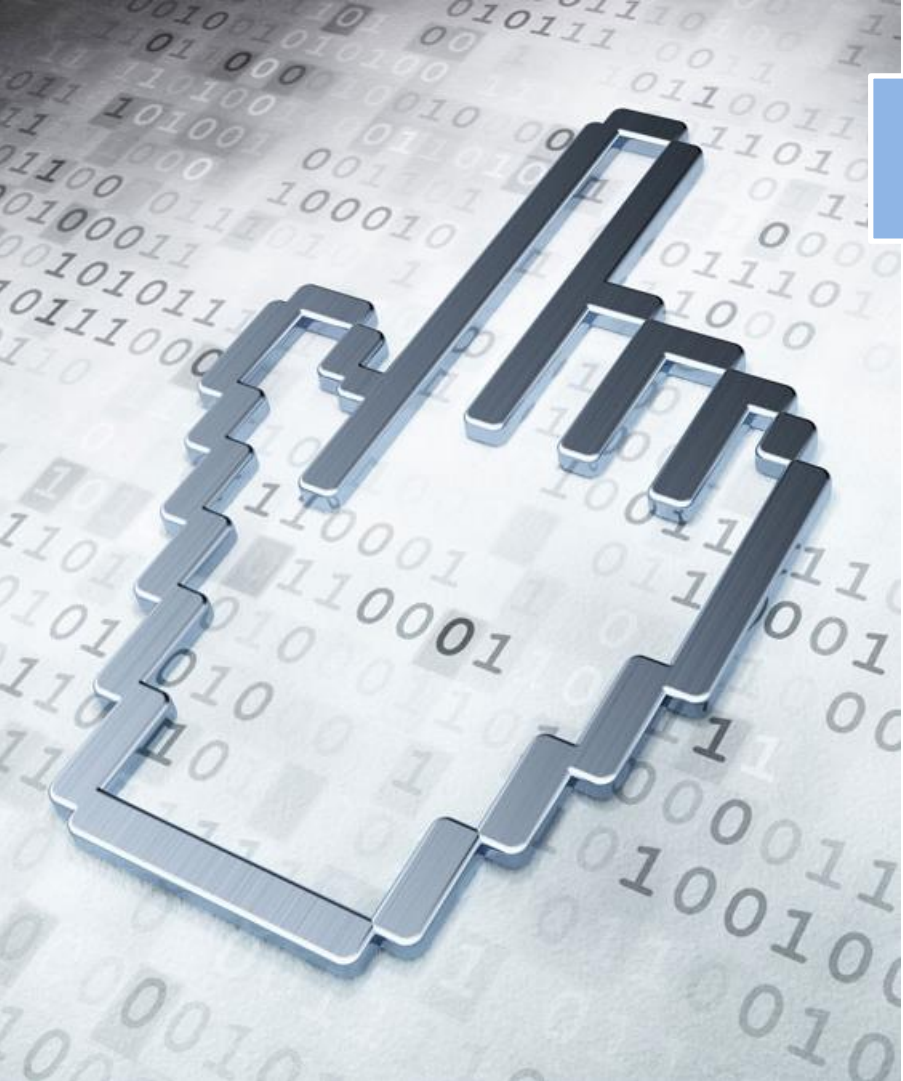
```
z = (int) t3
```

## 6.3.2 类型转换

➤ 加入类型检查的  $E \rightarrow E_1 + E_2$  的SDT

为语法成分设置 *type* 属性

```
{ E.addr := newtemp()
if  $E_1.type = integer$  and  $E_2.type = integer$  then begin
    gencode( E.addr, ':=' ,  $E_1.addr$ , 'int+',  $E_2.addr$  ) ;
     $E.type = integer$ 
end
else if  $E_1.type = integer$  and  $E_2.type = float$  then begin
     $u := newtemp()$ ;
    gencode(  $u$ , ':=' , '(float)',  $E_1.addr$  ) ;
    gencode( E.addr, ':=' ,  $u$ , 'float+',  $E_2.addr$  ) ;
     $E.type := float$ 
end ... }
```



# 本章内容

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 类型检查

**6.4 控制语句的翻译**

6.5 回填

6.6 switch语句的翻译

6.7 过程调用语句的翻译

## 6.4 控制语句的翻译

### ➤ 控制流语句的基本文法

➤  $P \rightarrow S$

➤  $S \rightarrow S_1 S_2$

➤  $S \rightarrow \text{id} = E ; \mid L = E ;$

➤  $S \rightarrow \text{if } B \text{ then } S_1$   
     $\mid \text{if } B \text{ then } S_1 \text{ else } S_2$   
     $\mid \text{while } B \text{ do } S_1$



# 控制流语句的代码结构



$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$

例: if  $a < b$  then  $y=0$  else  $y=1$

```
1:   if a < b goto L1  / B
2:   goto L2
3:  L1: y=0  ——— S1
4:     goto L3
5:  L2: y=1  ——— S2
6:  L3:
```

难点: 确定跳转指令中目标地址

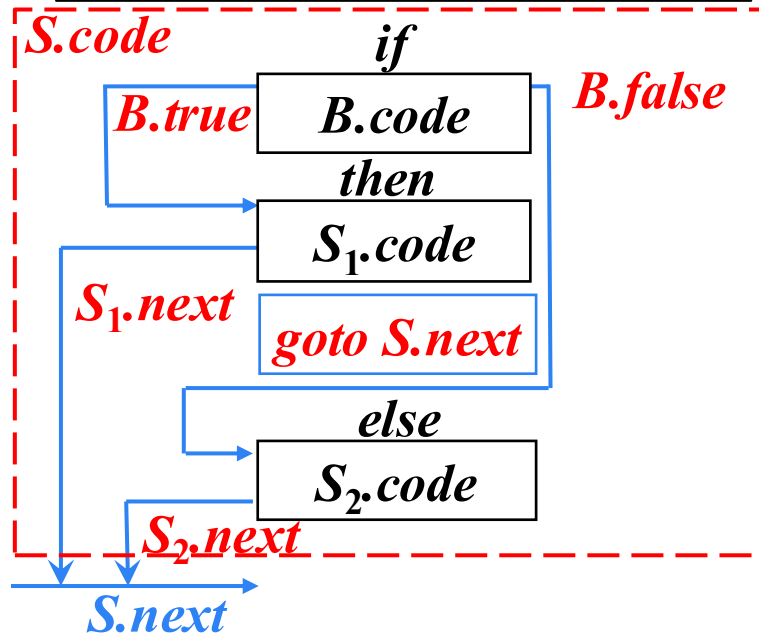
- 第一种方法: 为跳转目标预分配标号并通过继承属性传递到标注位置
  - 需第二遍扫描绑定标号与地址
- 第二种方法: 回填技术
  - 一遍扫描完成

布尔表达式 $B$ 被翻译成由跳转指令构成的跳转代码

# 控制流语句的代码结构

➤ 例

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



布尔表达式*B*被翻译成由  
跳转指令构成的跳转代码

➤ 继承属性

- *B.true*: 用来存放当*B*为真时控制流转向的指令的标号
- *B.false*: 用来存放当*B*为假时控制流转向的指令的标号
- *S.next*: 用来存放紧跟在*S*代码之后的指令(*S*的后继指令)的标号

用指令的标号标识一条三地址指令

## 控制流语句的SDT

***newlabel()***: 生成一个新指令标号并返回

➤  $P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$

➤  $S \rightarrow \{ S_1.next = newlabel(); \} S_1$

$\{ label(S_1.next); S_2.next = S.next; \} S_2$

***label(L)***: 将标号 $L$ 附加到下一条三地址指令前

➤  $S \rightarrow id = E ; \mid L = E ;$

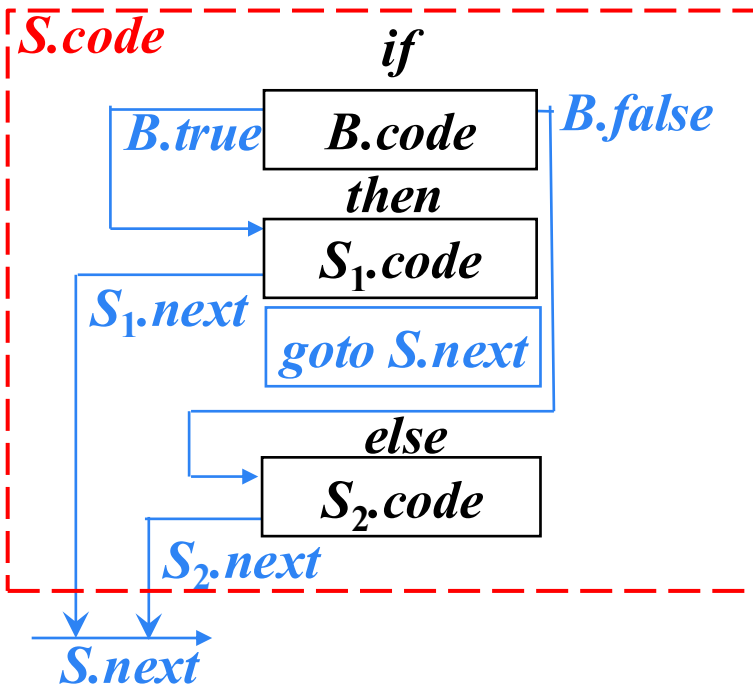
➤  $S \rightarrow if\ B\ then\ S_1$

$\mid if\ B\ then\ S_1\ else\ S_2$

$\mid while\ B\ do\ S_1$

# if-then-else语句的SDT

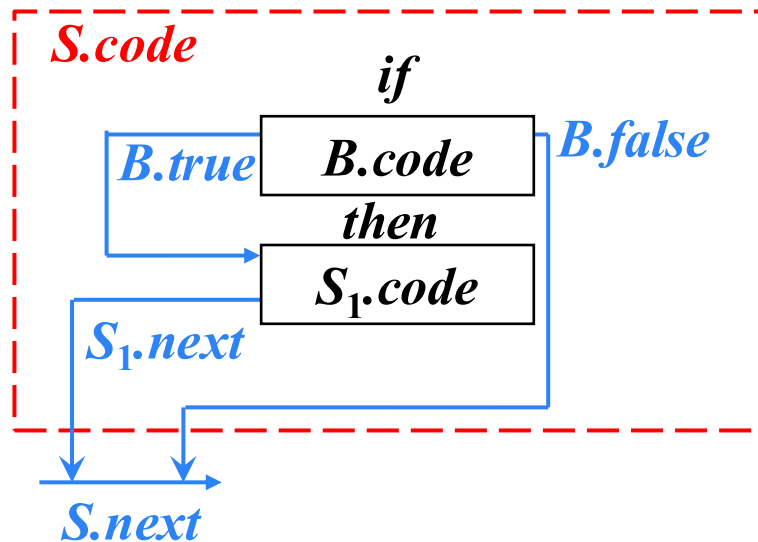
$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = \text{newlabel}(); \} B$   
 $\quad \text{then } \{ \text{label}(B.true); S_1.next = S.next; \} S_1 \{ \text{gen}( 'goto' S.next ) \}$   
 $\quad \text{else } \{ \text{label}(B.false); S_2.next = S.next; \} S_2$

## *if-then*语句的SDT

$S \rightarrow \text{if } B \text{ then } S_1$

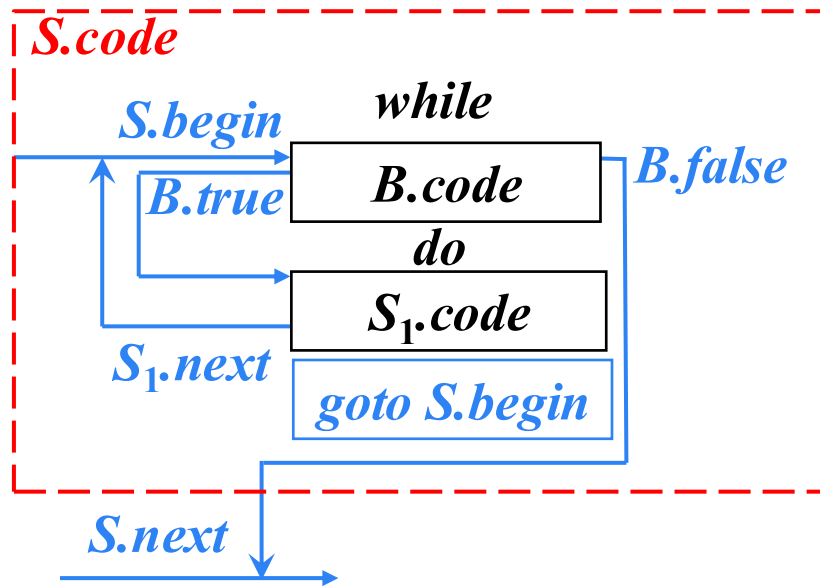


$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = S.next; \} B$   
 $\text{then } \{ \text{label}(B.true); S_1.next = S.next; \} S_1$

## *while-do*语句的SDT

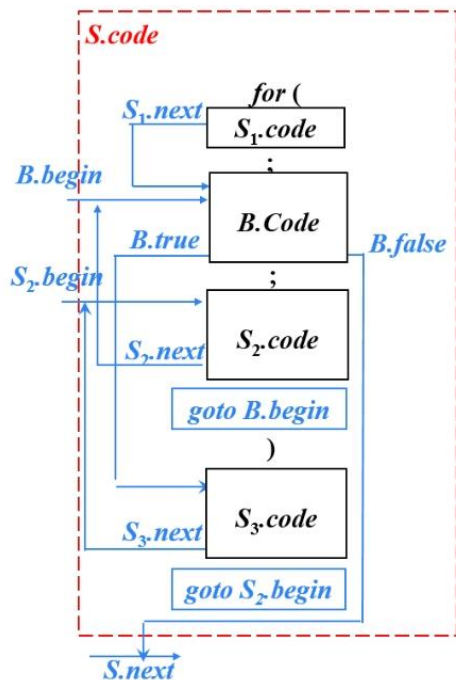
$S \rightarrow \text{while } B \text{ do } S_1$

$S \rightarrow \text{while } \{S.begin = \text{newlabel}();$   
 $\text{label}(S.begin);$   
 $B.true = \text{newlabel}();$   
 $B.false = S.next;\} B$   
 $\text{do } \{ \text{label}(B.true); S_1.next = S.begin;\} S_1$   
 $\{ \text{gen('goto' } S.begin); \}$



## 练习2

- 根据给定的代码结构图，参考6.4控制流语法的翻译方案，构造预标号的SDT，实现for控制流的翻译。



# 布尔表达式的翻译

## ➤ 布尔表达式的基本文法

$B \rightarrow B \text{ or } B$

|  $B \text{ and } B$

|  $\text{not } B$

|  $(B)$

|  $E \text{ relop } E$

|  $\text{true}$

|  $\text{false}$

优先级:  $\text{not} > \text{and} > \text{or}$

关系表达式

**relop (关系运算符) :**

$<, <=, >, >=, ==, !=$



# 布尔表达式的翻译

- 逻辑运算符&&、|| 和! 被翻译成**跳转指令**。运算符本身**不出现在代码中**，布尔表达式的值是通过代码序列中的**跳转位置**来表示的

- 例

```
if ( x<100 || x>200 && x!=y )  
    x=0;
```

- 语句

```
if x<100 goto L2  
goto L3
```

- 三地址代码

```
L3: if x>200 goto L4  
goto L1
```

```
L4: if x!=y goto L2  
goto L1
```

```
L2: x=0
```

```
L1:
```

## 布尔表达式的SDT

- $B \rightarrow E_1 \text{ relop } E_2 \{ \text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$   
 $\text{gen}(\text{'goto' } B.\text{false}); \}$
- $B \rightarrow \text{true} \{ \text{gen}(\text{'goto' } B.\text{true}); \}$
- $B \rightarrow \text{false} \{ \text{gen}(\text{'goto' } B.\text{false}); \}$
- $B \rightarrow ( \{ B_1.\text{true} = B.\text{true}; B_1.\text{false} = B.\text{false}; \} B_1 )$
- $B \rightarrow \text{not} \{ B_1.\text{true} = B.\text{false}; B_1.\text{false} = B.\text{true}; \} B_1$

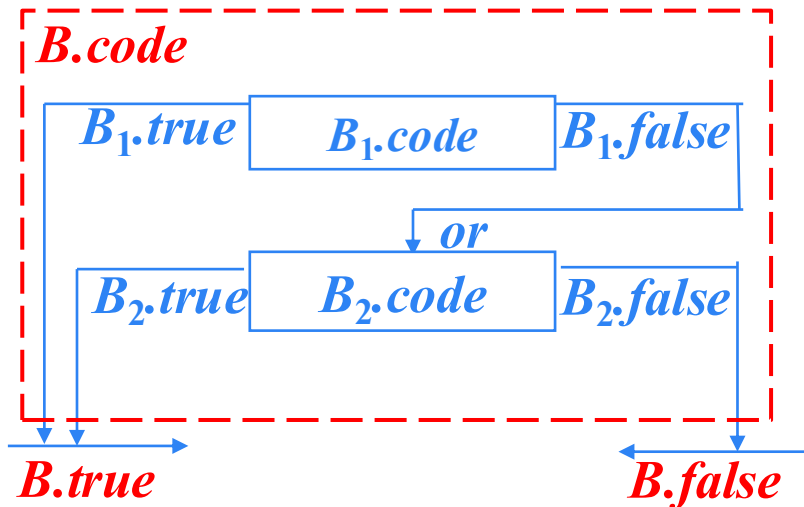
## $B \rightarrow B_1 \text{ or } B_2$ 的SDT

➤  $B \rightarrow B_1 \text{ or } B_2$

➤  $B \rightarrow \{ B_1.true = B.true; B_1.false = \text{newlabel}(); \} B_1$

or  $\{ \text{label}(B_1.false); B_2.true = B.true; B_2.false = B.false; \} B_2$

生成短路代码

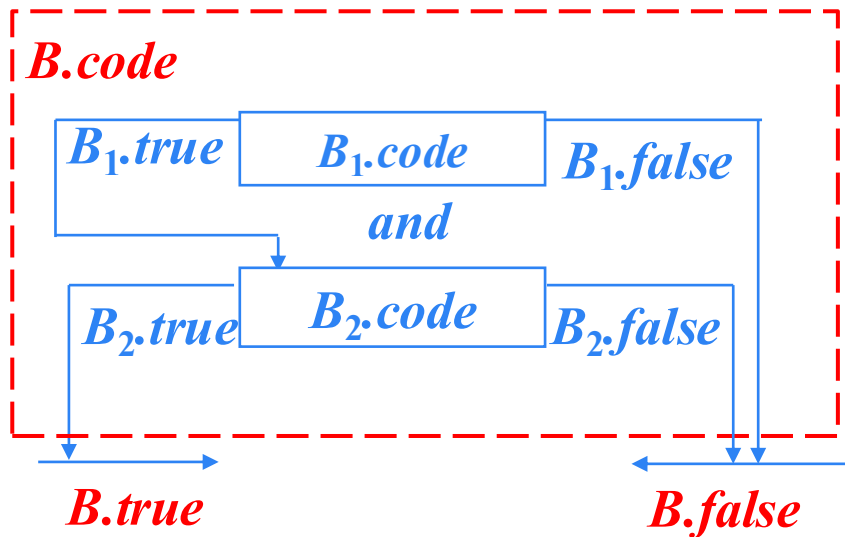


## $B \rightarrow B_1 \text{ and } B_2$ 的SDT

➤  $B \rightarrow B_1 \text{ and } B_2$

➤  $B \rightarrow \{ B_1.\text{true} = \text{newlabel}(); B_1.\text{false} = B.\text{false}; \} B_1$   
and  $\{ \text{label}(B_1.\text{true}); B_2.\text{true} = B.\text{true}; B_2.\text{false} = B.\text{false}; \} B_2$

生成短路代码



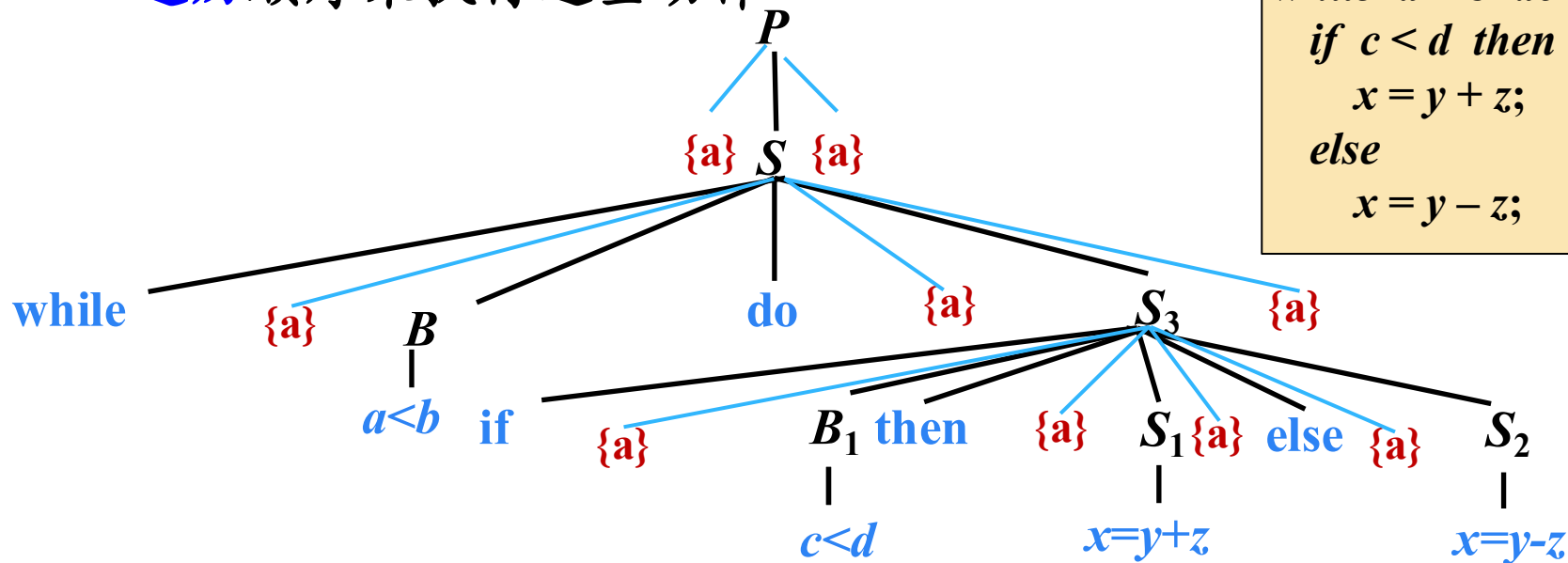
# 控制流语句的SDT

- $P \rightarrow \{a\}S\{a\}$
- $S \rightarrow \{a\}S_1\{a\}S_2$
- $S \rightarrow \text{id}=E;\{a\} \mid L=E;\{a\}$
- $E \rightarrow E_1+E_2\{a\} \mid -E_1\{a\} \mid (E_1)\{a\} \mid \text{id}\{a\} \mid L\{a\}$
- $L \rightarrow \text{id}[E]\{a\} \mid L_1[E]\{a\}$
- $S \rightarrow \text{if } \{a\}B \text{ then } \{a\}S_1$ 
  - $\mid \text{if } \{a\}B \text{ then } \{a\}S_1 \{a\} \text{ else } \{a\}S_2$
  - $\mid \text{while } \{a\}B \text{ do } \{a\}S_1\{a\}$
- $B \rightarrow \{a\}B \text{ or } \{a\}B \mid \{a\}B \text{ and } \{a\}B \mid \text{not } \{a\}B \mid (\{a\}B)$ 
  - $\mid E \text{ relop } E\{a\} \mid \text{true}\{a\} \mid \text{false}\{a\}$

```
while  $a < b$  do  
    if  $c < d$  then  
         $x = y + z;$   
    else  
         $x = y - z;$ 
```

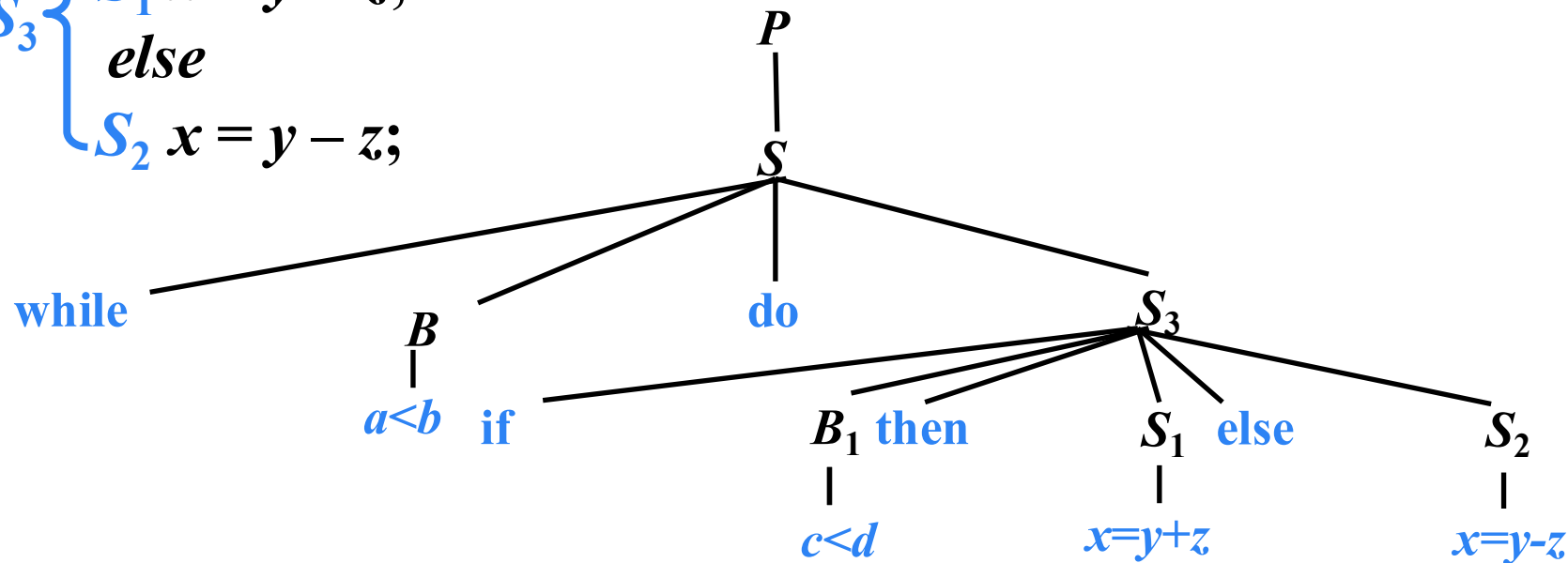
# SDT的通用实现方法

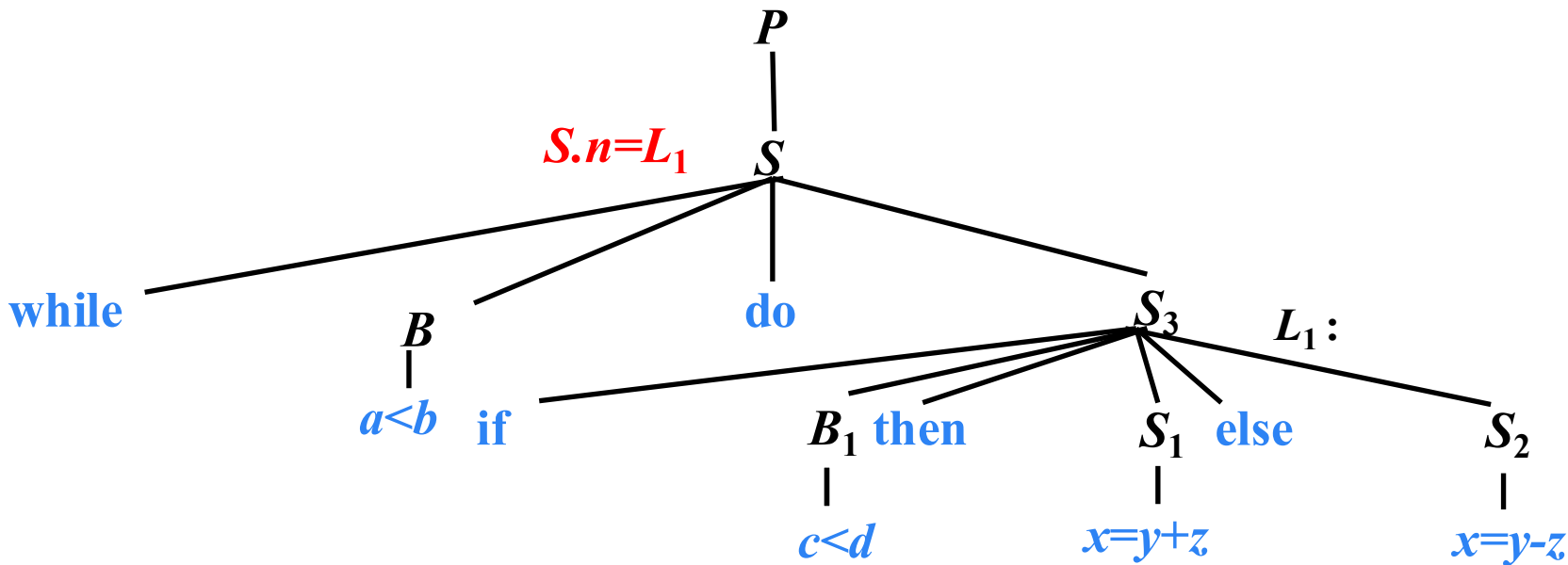
- 任何SDT都可以通过下面的方法实现
- 首先建立一棵语法分析树，然后按照**从左到右**的**深度优先遍历**顺序来执行这些动作



# 例

$B$   
 $while\ a < b\ do$   
 $B_1$   
 $S_3$  {  $S_1\ x = y + z;$   
 $else$   
 $S_2\ x = y - z;$




$$P \rightarrow \{ \textcolor{blue}{S.next = newlabel();} \} S \{ \textcolor{red}{label(S.next);} \}$$




# 例

$L_2 : \text{if } a < b \text{ goto } L_3$   
 $\text{goto } L_1$

$L_3 :$

$B \rightarrow E_1 \text{ relop } E_2$   
 $\{ \text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$   
 $\text{gen}(\text{'goto' } B.\text{false}); \}$

$S \rightarrow \text{while } \{ S.\text{begin} = \text{newlabel}();$

$\text{label}(S.\text{begin});$

$B.\text{true} = \text{newlabel}());$

$B.\text{false} = S.\text{next}; \}$   $B$

$\text{do } \{ \text{label}(B.\text{true});$

$S_1.\text{next} = S.\text{begin}; \}$   $S_1$

$\{ \text{gen}(\text{'goto' } S.\text{begin}); \}$

$S.n = L_1$

$\text{while}$

$B.t = L_3$   
 $B.f = L_1$

$a < b$

$\text{if}$

$\text{do}$

$S_3.n = L_2$

$S_3$

$L_1 :$

$\text{goto } L_2$

$B_1$

$\text{then}$

$S_1$

$\text{else}$

$S_2$

$c < d$

$x = y + z$

$x = y - z$

$S.\text{begin} = L_2$


**例**
$$S \rightarrow \text{if } \{ B.\text{true} = \text{newlabel}(); B.\text{false} = \text{newlabel}(); \} B$$
$$\text{then } \{ \text{label}(B.\text{true}); S_1.\text{next} = S.\text{next}; \} S_1$$
$$\{ \text{gen}(\text{'goto' } S.\text{next}); \}$$
$$\text{else } \{ \text{label}(B.\text{false});$$
$$S_2.\text{next} = S.\text{next}; \} S_2$$
$$P$$
$$|$$

**$L_2$  : if  $a < b$  goto  $L_3$   
goto  $L_1$**

***$L_3$ : if  $c < d$  goto  $L_4$   
goto  $L_5$***

```

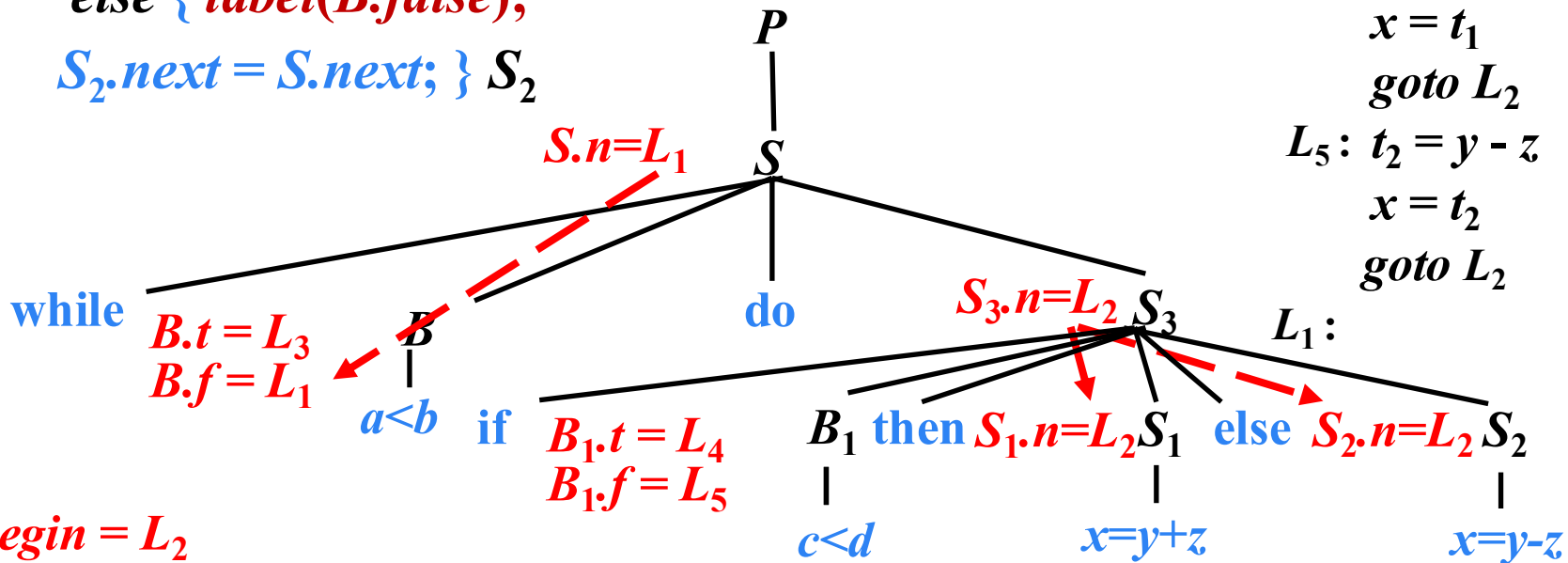
L4: t1 = y + z
      x = t1
      goto L2

```

```

L5: t2 = y - z
      x = t2
      goto L2

```



## 语句 “*while* $a < b$ *do if* $c < d$ *then* $x = y + z$ *else* $x = y - z$ ” 的三地址代码

1:  $L_2 : \text{if } a < b \text{ goto } L_3$   
2:  $\text{goto } L_1$   
3:  $L_3 : \text{if } c < d \text{ goto } L_4$   
4:  $\text{goto } L_5$   
5:  $L_4 : t_1 = y + z$   
6:  $x = t_1$   
7:  $\text{goto } L_2$   
8:  $L_5 : t_2 = y - z$   
9:  $x = t_2$   
10:  $\text{goto } L_2$   
11:  $L_1 :$

需要第二趟扫描  
绑定标号与  
指令序号

1: ( $j <$ ,  $a$ ,  $b$ , 3)  
2: ( $j$ , -, -, 11)  
3: ( $j <$ ,  $c$ ,  $d$ , 5)  
4: ( $j$ , -, -, 8)  
5: (+,  $y$ ,  $z$ ,  $t_1$ )  
6: (=,  $t_1$ , -,  $x$ )  
7: ( $j$ , -, -, 1)  
8: (-,  $y$ ,  $z$ ,  $t_2$ )  
9: (=,  $t_2$ , -,  $x$ )  
10: ( $j$ , -, -, 1)  
11:

# 语句 “*while* $a < b$ *do if* $c < d$ *then* $x = y + z$ *else* $x = y - z$ ” 的三地址代码

*B.first* → 1: *if*  $a < b$  *goto* 3

2: *goto* 11

*S<sub>1</sub>.first* → 3: *if*  $c < d$  *goto* 5

4: *goto* 8

5:  $t_1 = y + z$

6:  $x = t_1$

7: *goto* 1

8:  $t_2 = y - z$

9:  $x = t_2$

10: *goto* 1

11:

1: *ifFalse*  $a < b$  *goto* 11

2:

3: *ifFalse*  $c < d$  *goto* 8

4:

5:  $t_1 = y + z$

6:  $x = t_1$

7: *goto* 1

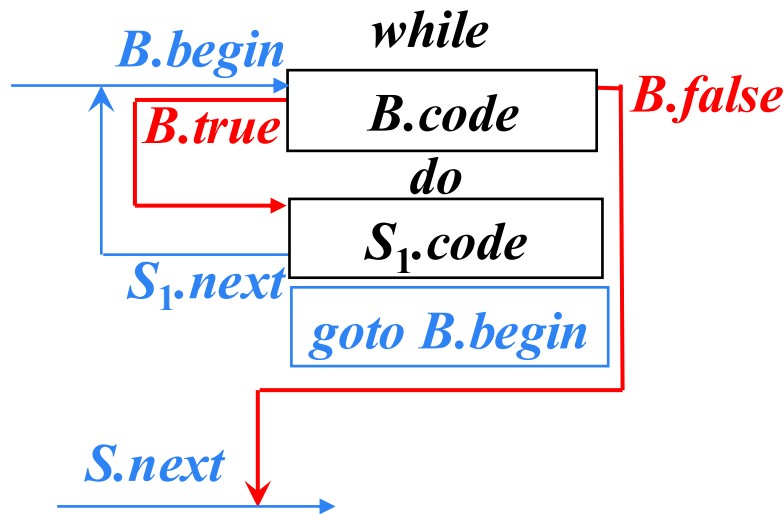
8:  $t_2 = y - z$

9:  $x = t_2$

10: *goto* 1

11:

如何避免生成冗余的goto指令?



# 语句 “*while* $a < b$ *do if* $c < d$ *then* $x = y + z$ *else* $x = y - z$ ” 的三地址代码

1: *if*  $a < b$  *goto* 3

2: *goto* 11

*B.first* → 3: *if*  $c < d$  *goto* 5

4: *goto* 8

*S<sub>1</sub>.first* → 5:  $t_1 = y + z$

6:  $x = t_1$

7: *goto* 1

*S<sub>2</sub>.first* → 8:  $t_2 = y - z$

9:  $x = t_2$

10: *goto* 1

11:

1: *ifFalse*  $a < b$  *goto* 11

2:

3: *ifFalse*  $c < d$  *goto* 8

4:

5:  $t_1 = y + z$

6:  $x = t_1$

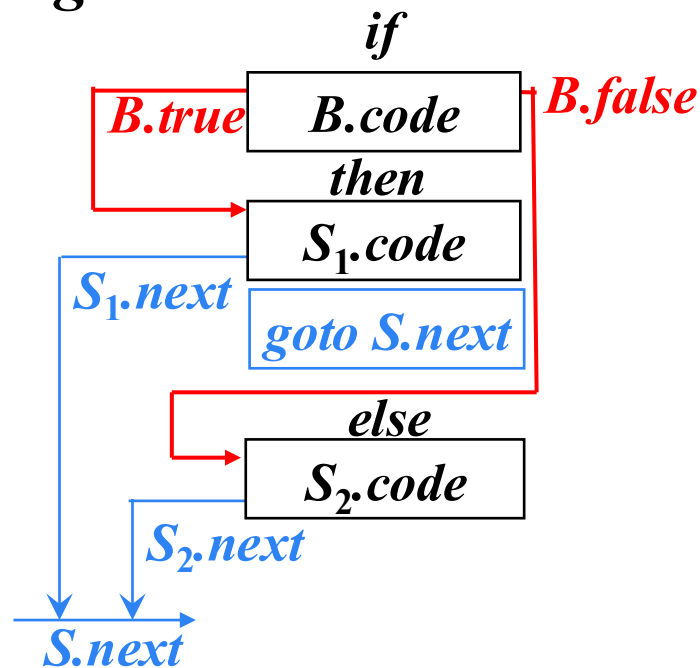
7: *goto* 1

8:  $t_2 = y - z$

9:  $x = t_2$

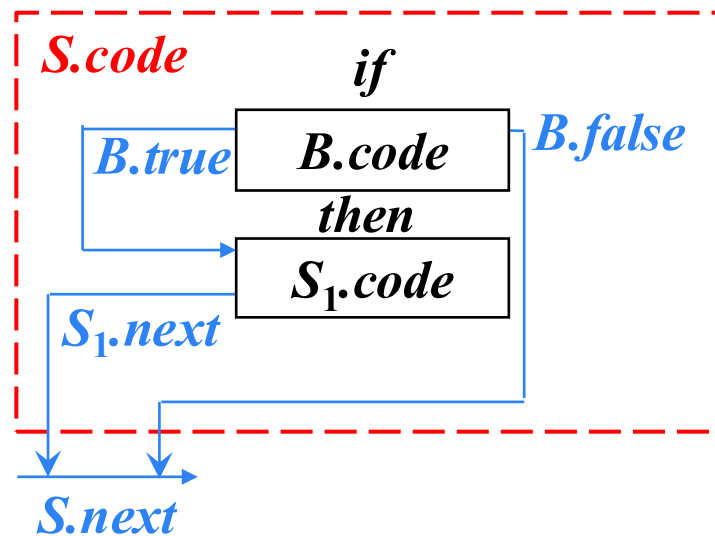
10: *goto* 1

11:



# 避免生成冗余的goto指令

## 修改if-then语句的SDT



$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = S.next; \} B$   
 $\text{then } \{ \text{label}(B.true); S_1.next = S.next; \} S_1$

不要生成任何跳转指令



$S \rightarrow \text{if } \{ B.true = \text{fall}; B.false = S.next; \} B$   
 $\text{then } \{ S_1.next = S.next; \} S_1$

## 修改 $B \rightarrow E_1 \text{ relop } E_2$ 的SDT

➤  $B \rightarrow E_1 \text{ relop } E_2 \{ \text{gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$   
 $\text{gen('goto' } B.\text{false}); \}$



➤  $B \rightarrow E_1 \text{ relop } E_2$

{ if  $B.\text{true} \neq \text{fall}$  and  $B.\text{false} \neq \text{fall}$  then

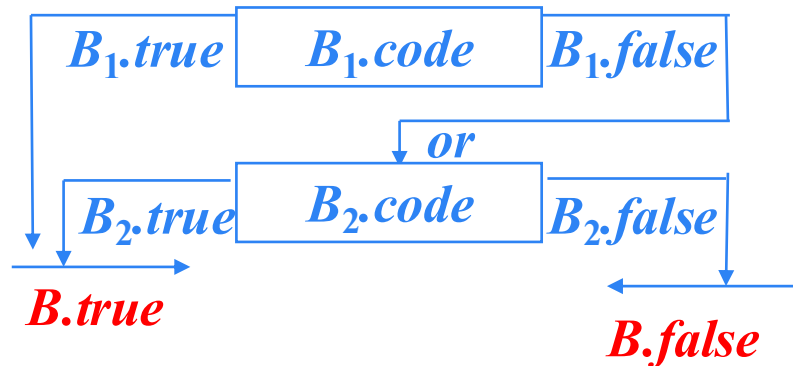
$\text{gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true}); \text{ gen('goto' } B.\text{false}); \}$

else if  $B.\text{true} \neq \text{fall}$  then  $\text{gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$

else if  $B.\text{false} \neq \text{fall}$  then  $\text{gen('ifFalse' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{false});$

else ' ' }

## 修改 $B \rightarrow B_1 \text{ or } B_2$ 的SDT



$B \rightarrow \{ B_1.true = B.true; B_1.false = newlabel(); \} B_1$   
or  $\{ label(B_1.false); B_2.true = B.true; B_2.false = B.false; \} B_2$

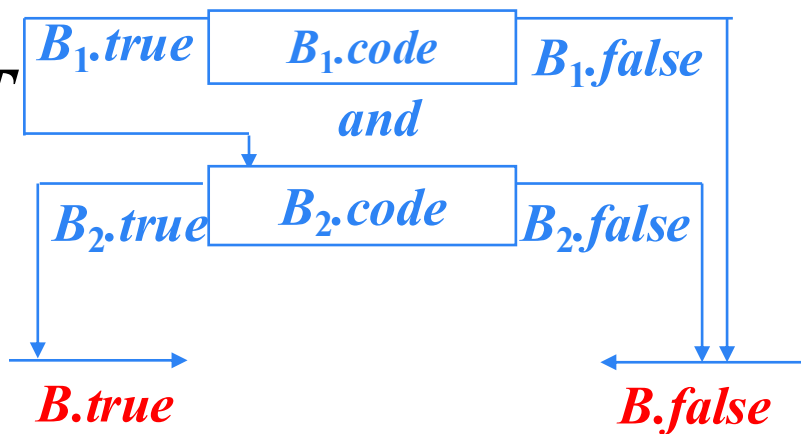
↓

$B \rightarrow \{ B_1.true = \text{if } B.true \neq fall \text{ then } B.true \text{ else } newlabel(); B_1.false = fall; \} B_1$   
or  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ label(B_1.true); \}$





修改  $B \rightarrow B_1$  and  $B_2$  的SDT



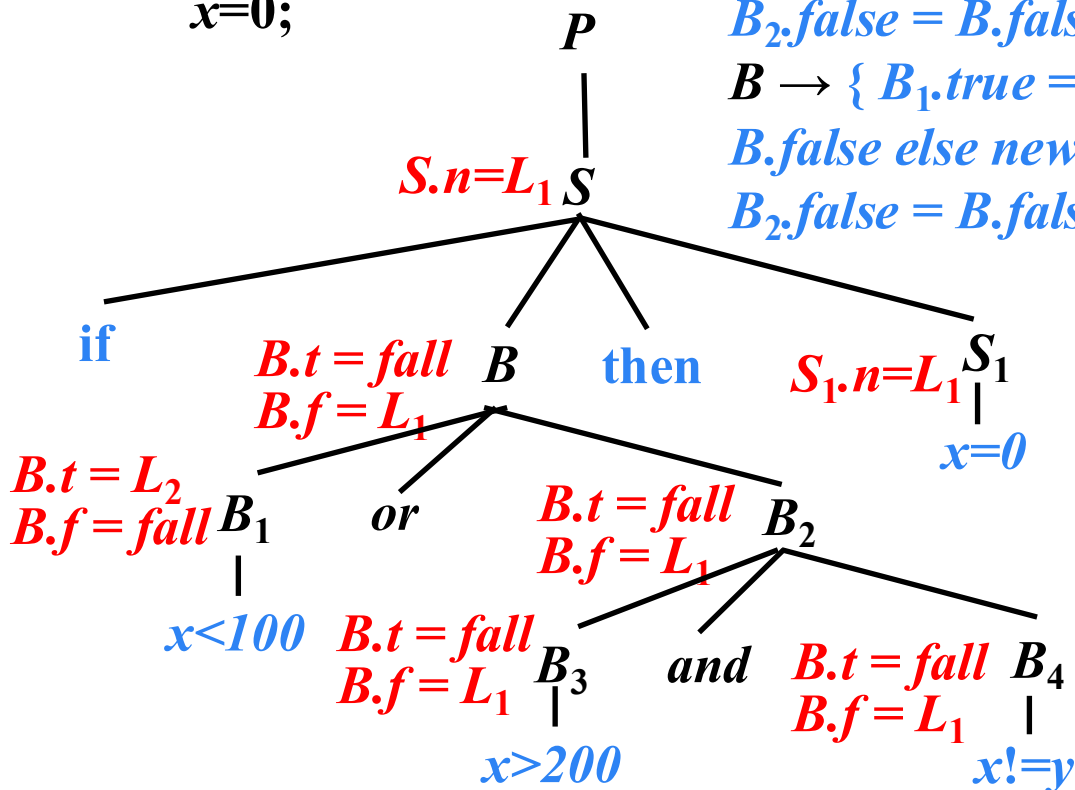
$B \rightarrow \{ B_1.true = newlabel(); B_1.false = B.false; \} B_1$   
 and  $\{ label(B_1.true); B_2.true = B.true; B_2.false = B.false; \} B_2$



$B \rightarrow \{ B_1.true = fall; B_1.false = if B.false \neq fall then B.false else newlabel(); \} B_1$   
 and  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ label(B_1.false ); \}$

# 例

*if* (  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$  )  
 $x = 0;$



$S \rightarrow \text{if } \{ B.true = \text{fall}; B.false = S.next; \} B$   
 $\text{then } \{ S_1.next = S.next; \} S_1$

$B \rightarrow \{ B_1.true = \text{if } B.true \neq \text{fall} \text{ then } B.true \text{ else } \text{newlabel}(); B_1.false = \text{fall}; \} B_1$  **or**  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ \text{label}( B_1.true ); \}$

$B \rightarrow \{ B_1.true = \text{fall}; B_1.false = \text{if } B.false \neq \text{fall} \text{ then } B.false \text{ else } \text{newlabel}(); \} B_1$  **and**  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ \text{label}( B_1.false ); \}$

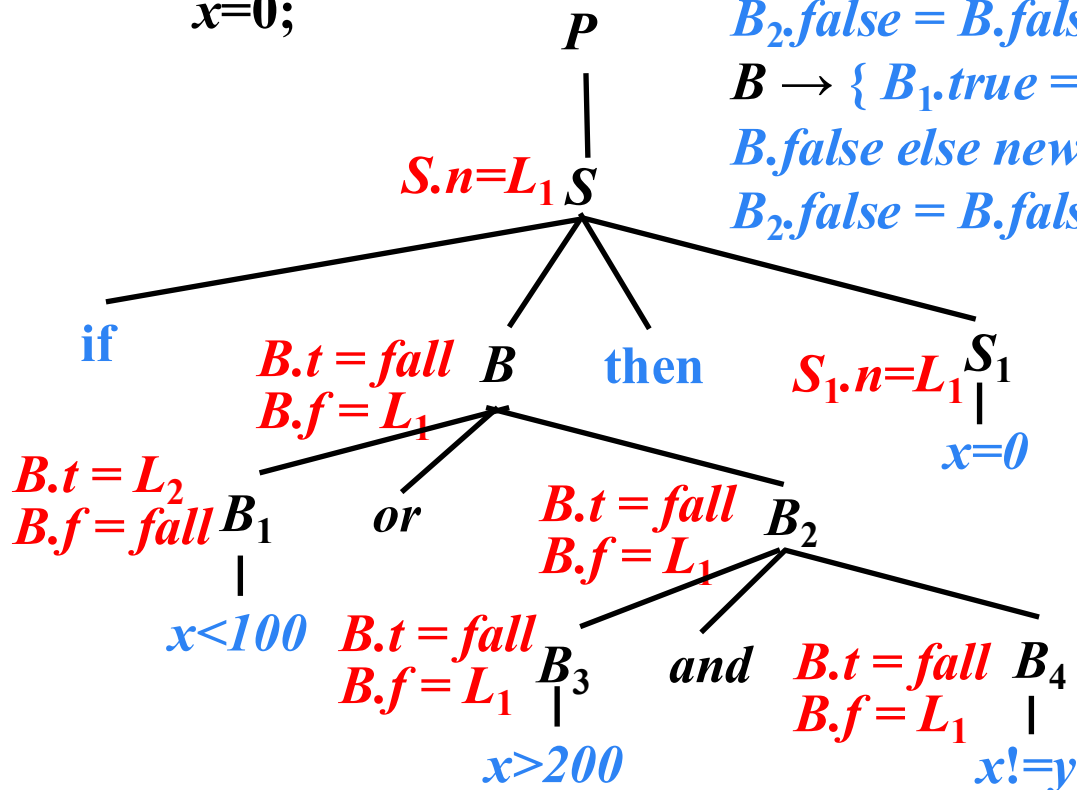
*if*  $x < 100$  *goto*  $L_2$   
*ifFalse*  $x > 200$  *goto*  $L_1$   
*ifFalse*  $x \neq y$  *goto*  $L_1$

$L_1 \ L_2 : \ x = 0$

# 例

*if* (  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$  )

$x = 0;$



$S \rightarrow \text{if } \{ B.true = \text{fall}; B.false = S.next; \} B$   
 $\text{then } \{ S_1.next = S.next; \} S_1$

$B \rightarrow \{ B_1.true = \text{if } B.true \neq \text{fall} \text{ then } B.true \text{ else } \text{newlabel}(); B_1.false = \text{fall}; \} B_1$  **or**  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ \text{label}( B_1.true ); \}$

$B \rightarrow \{ B_1.true = \text{fall}; B_1.false = \text{if } B.false \neq \text{fall} \text{ then } B.false \text{ else } \text{newlabel}(); \} B_1$  **and**  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ \text{label}( B_1.false ); \}$

*if*  $x < 100$  *goto*  $L_2$   
*ifFalse*  $x > 200$  *goto*  $L_1$   
*ifFalse*  $x \neq y$  *goto*  $L_1$

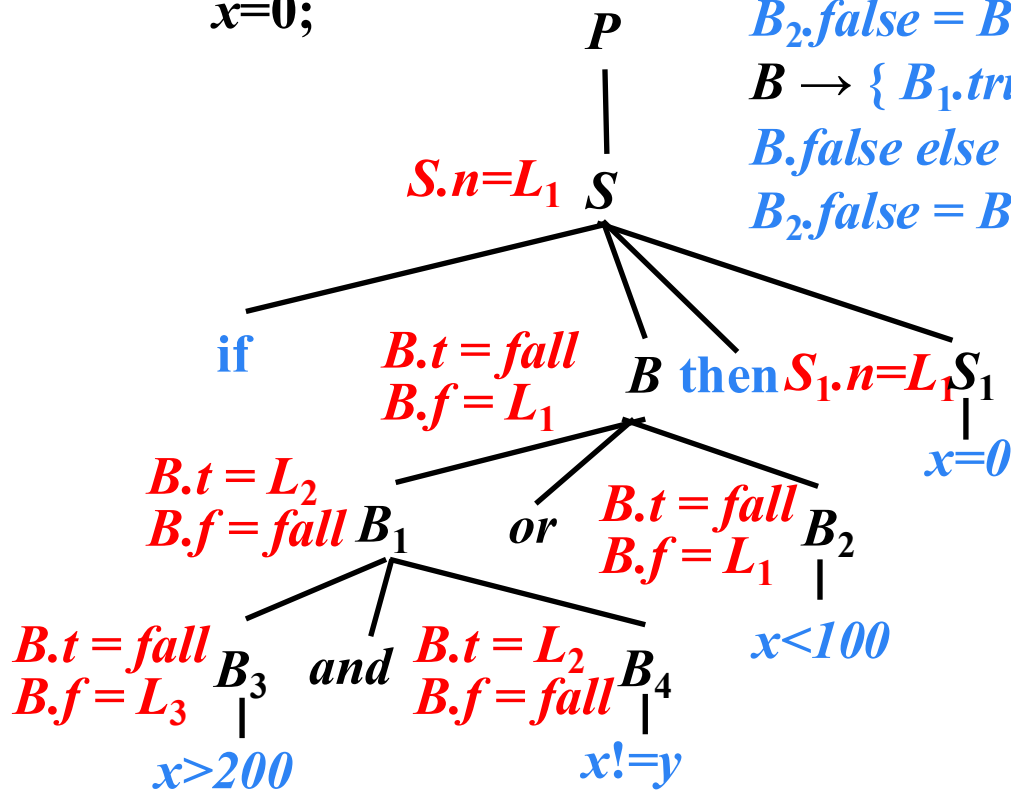
$L_2 :$   $x = 0$

$L_1 :$

# 例

*if* (*x*>200 && *x*!=*y* || *x*<100 )

*x*=0;



$S \rightarrow \text{if } \{ B.true = \text{fall}; B.false = S.next; \} B$   
 $\text{then } \{ S_1.next = S.next; \} S_1$

$B \rightarrow \{ B_1.true = \text{if } B.true \neq \text{fall} \text{ then } B.true \text{ else } \text{newlabel}(); B_1.false = \text{fall}; \} B_1$  **or**  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ \text{label}( B_1.true ); \}$

$B \rightarrow \{ B_1.true = \text{fall}; B_1.false = \text{if } B.false \neq \text{fall} \text{ then } B.false \text{ else } \text{newlabel}(); \} B_1$  **and**  $\{ B_2.true = B.true; B_2.false = B.false; \} B_2 \{ \text{label}( B_1.false ); \}$

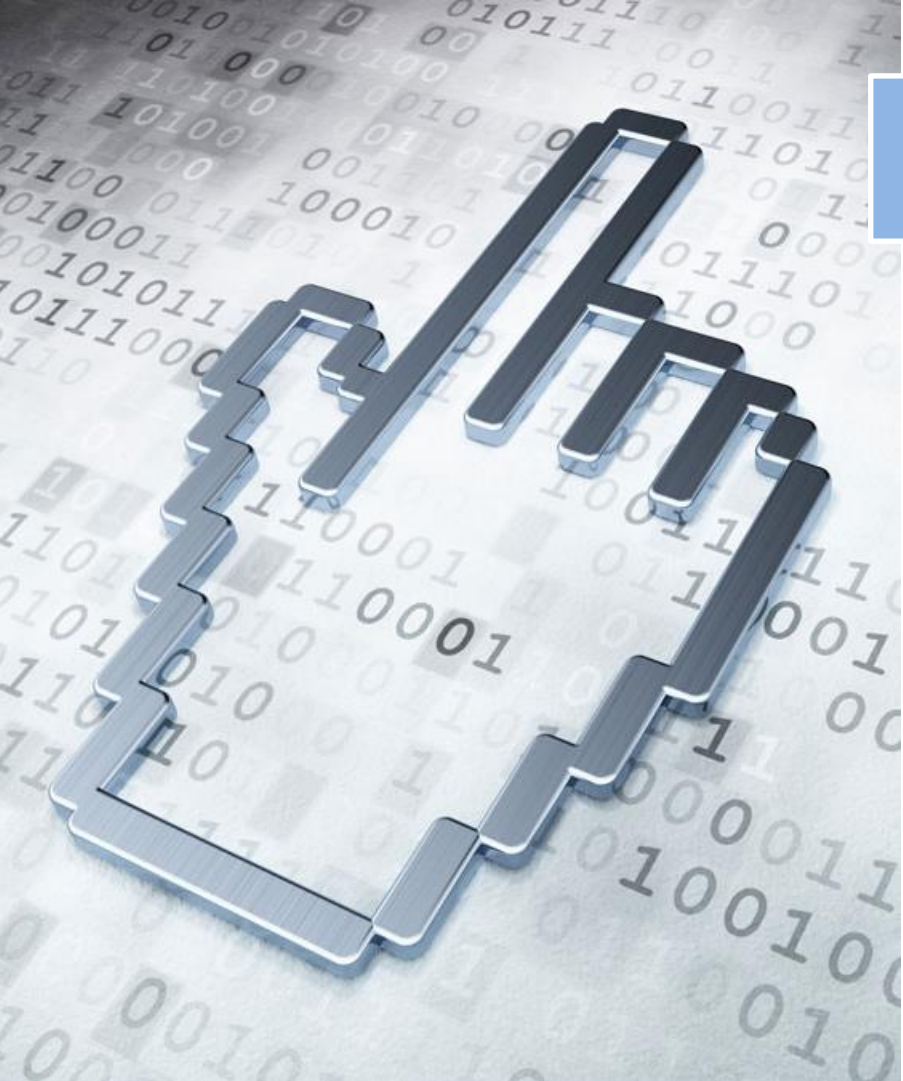
*ifFalse* *x*>200 *goto* *L*<sub>3</sub>

*if* *x*!=*y* *goto* *L*<sub>2</sub>

*L*<sub>3</sub> : *ifFalse* *x*<100 *goto* *L*<sub>1</sub>

*L*<sub>2</sub> : *x*=0

*L*<sub>1</sub> :



# 本章内容

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 类型检查

6.4 控制语句的翻译

**6.5 回填**

6.6 switch语句的翻译

6.7 过程调用语句的翻译

## 6.5 回填 (*Backpatching*)

### ➤ 基本思想

- 生成一个跳转指令时，暂时不指定该跳转指令的目标标号。具有相同的目标标号的跳转指令组成一个列表并以综合属性的形式进行传递。等到能够确定正确的目标标号时，才去填充这些指令的目标标号。
- 优点：在一趟扫描中即可完成跳转语句生成与目标标号的填充。

## 非终结符 $B$ 的综合属性

- $B.truelist$ : 指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是当  $B$  为真时控制流应该转向的指令的标号
  - $B.falselist$ : 指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是当  $B$  为假时控制流应该转向的指令的标号
- 将生成的指令按顺序编号，指令的序号作为标号

# 函数

## ➤ *makelist( i )*

- 创建一个只包含 $i$ 的列表， $i$ 是跳转指令的标号，函数返回指向新创建的列表的指针

## ➤ *merge( $p_1, p_2$ )*

- 将 $p_1$ 和 $p_2$ 指向的列表进行合并，返回指向合并后的列表的指针

## ➤ *backpatch( $p, j$ )*

- 将 $j$ 作为目标标号插入到 $p$ 所指列表中的各跳转指令中



## 布尔表达式的回填

➤  $B \rightarrow E_1 \text{ relop } E_2$

{

*$B.\text{truelist} = \text{makelist}(\text{nextquad});$*

*$B.\text{falselist} = \text{makelist}(\text{nextquad}+1);$*

*$\text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto _'});$*

*$\text{gen}(\text{'goto _'});$*

}

*nextquad*: 即将生成的下一条指令的标号，也就是下一条跳转指令的标号

➤ 语义动作都在产生式末尾，可以在自底向上语法分析中实现的SDT

## 布尔表达式的回填

➤  $B \rightarrow E_1 \text{ relop } E_2$

➤  $B \rightarrow \text{true}$

{

*$B.\text{truelist} = \text{makelist}(\text{nextquad});$*

*$\text{gen}(\text{'goto _'});$*

}

# 布尔表达式的回填

➤  $B \rightarrow E_1 \text{ relop } E_2$

➤  $B \rightarrow \text{true}$

➤  $B \rightarrow \text{false}$

```
{  
    B.falselist = makelist(nextquad);  
    gen('goto _');  
}
```

# 布尔表达式的回填

➤  $B \rightarrow E_1 \text{ relop } E_2$

➤  $B \rightarrow \text{true}$

➤  $B \rightarrow \text{false}$

➤  $B \rightarrow (B_1)$

{

*$B.\text{truelist} = B_1.\text{truelist} ;$*

*$B.\text{falselist} = B_1.\text{falselist} ;$*

}

# 布尔表达式的回填

➤  $B \rightarrow E_1 \text{ relop } E_2$

➤  $B \rightarrow \text{true}$

➤  $B \rightarrow \text{false}$

➤  $B \rightarrow (B_1)$

➤  $B \rightarrow \text{not } B_1$

{

*$B.\text{truelist} = B_1.\text{falselist};$*

*$B.\text{falselist} = B_1.\text{truelist};$*

}

$B \rightarrow B_1 \text{ or } B_2$

$B \rightarrow B_1 \text{ or } M B_2$

{

$B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$

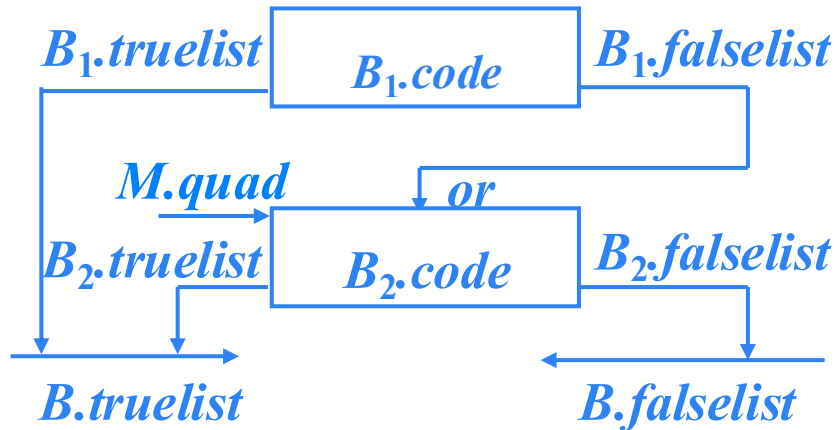
$B.\text{falselist} = B_2.\text{falselist};$

$\text{backpatch}(B_1.\text{falselist}, M.\text{quad});$

}

$M \rightarrow \varepsilon$

{  $M.\text{quad} = \text{nextquad};$  }



$B \rightarrow B_1 \text{ and } B_2$

$B \rightarrow B_1 \text{ and } M B_2$

{

$B.truelist = B_2.truelist;$

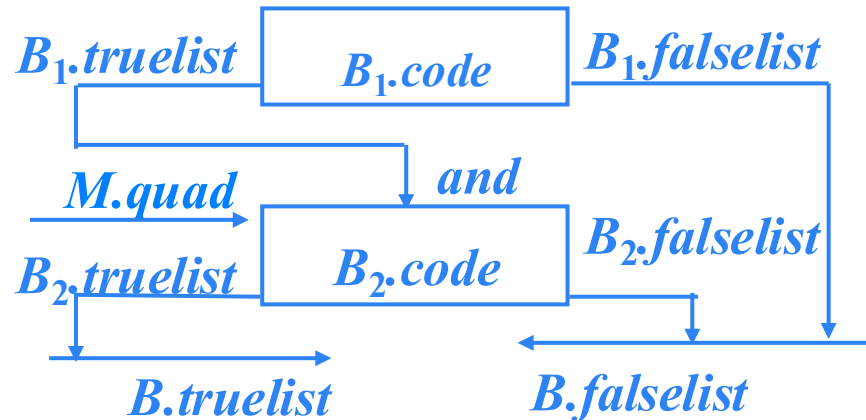
$B.falselist = merge( B_1.falselist, B_2.falselist );$

$backpatch( B_1.truelist, M.quad );$

}

$M \rightarrow \varepsilon$

{  $M.quad = nextquad ;$  }



# 例

$B \rightarrow E_1 \text{ relop } E_2$

```
{ B.truelist = makelist(nextquad);  
  B.falselist = makelist(nextquad+1);  
  gen('if'  $E_1.addr \text{ relop } E_2.addr$  'goto _');  
  gen('goto _');  
}
```

100: *if*  $a < b$  goto \_

101: goto \_

$t = \{100\}$   
 $f = \{101\}$   
 $B$   
 $a < b$  or  $c < d$  and  $e < f$



# 例

$B \rightarrow B_1 \text{ or } M B_2$

{

*backpatch(  $B_1$ .falselist,  $M$ .quad );*

*$B$ .truelist = merge(  $B_1$ .truelist,  $B_2$ .truelist );*

*$B$ .falselist =  $B_2$ .falselist ;*

}

$M \rightarrow \varepsilon$

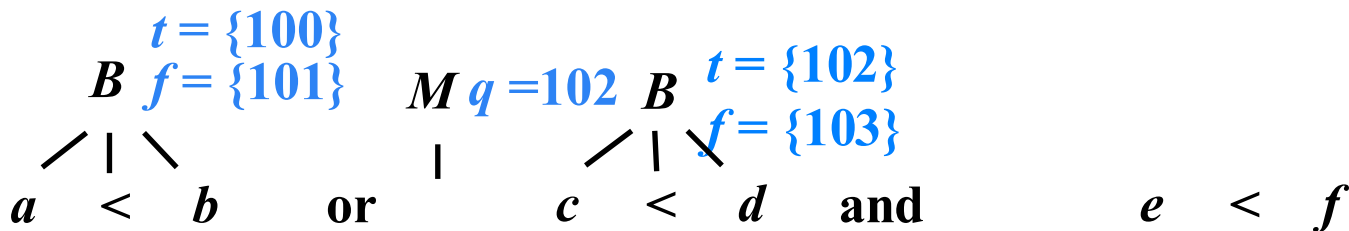
{  *$M$ .quad = nextquad ;* }

100: *if  $a < b$  goto \_*

101: *goto \_*

102: *if  $c < d$  goto \_*

103: *goto \_*



# 例

$B \rightarrow B_1 \text{ and } M B_2$

{

$B.\text{truelist} = B_2.\text{truelist};$

$B.\text{falselist} = \text{merge}( B_1.\text{falselist}, B_2.\text{falselist} );$

$\text{backpatch}( B_1.\text{truelist}, M.\text{quad} );$

}

100: *if*  $a < b$  *goto* \_

101: *goto* \_

102: *if*  $c < d$  *goto* 104

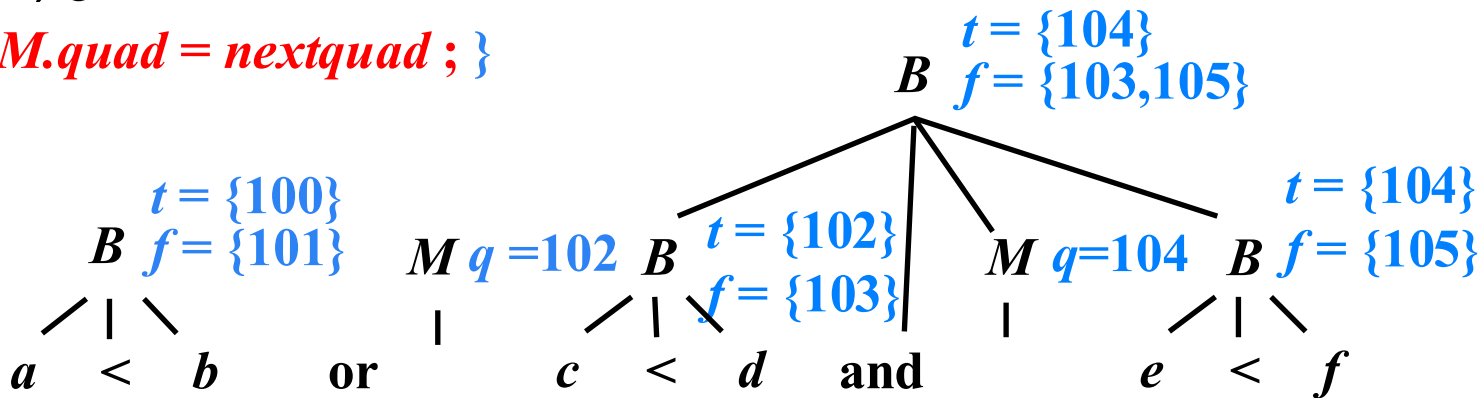
103: *goto* \_

104: *if*  $e < f$  *goto* \_

105: *goto* \_

$M \rightarrow \varepsilon$

{  $M.\text{quad} = \text{nextquad};$  }



# 例

$B \rightarrow B_1 \text{ or } M B_2$

{

$B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$

$B.\text{falselist} = B_2.\text{falselist};$

$\text{backpatch}(B_1.\text{falselist}, M.\text{quad});$

}

100: *if*  $a < b$  goto \_ *true*

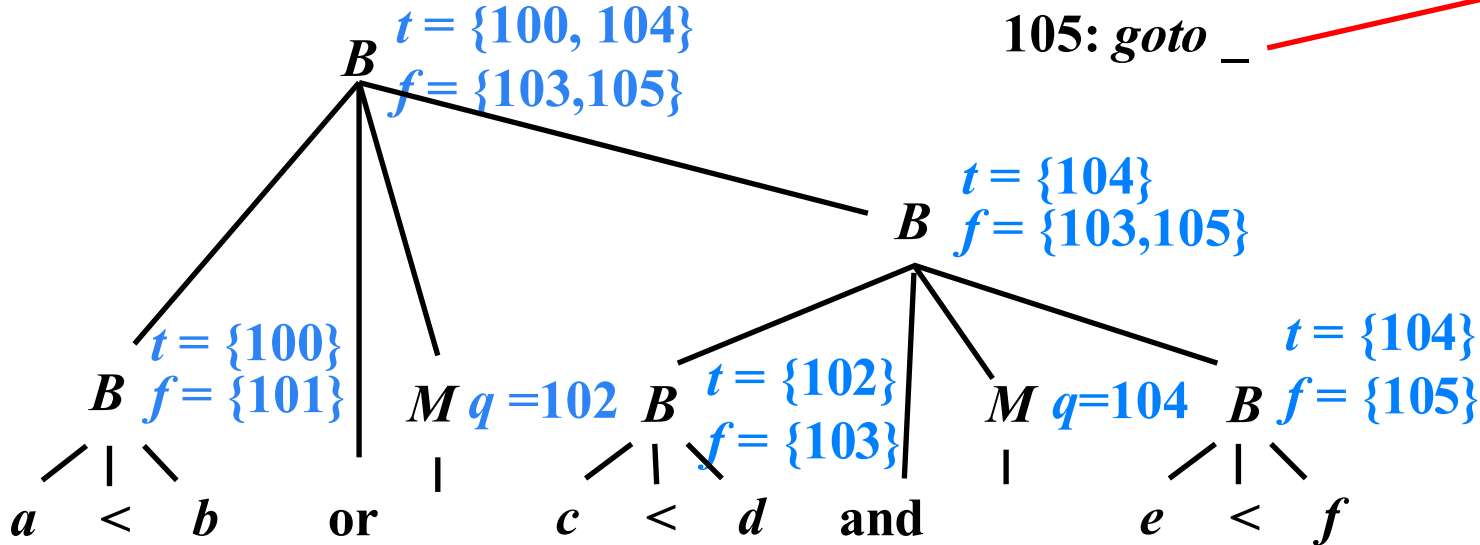
101: goto **102**

102: *if*  $c < d$  goto **104**

103: goto \_

104: *if*  $e < f$  goto \_ *false*

105: goto \_



# 控制流语句的回填

## ➤ 文法

➤  $S \rightarrow S_1 S_2$

➤  $S \rightarrow \text{id} = E ; \mid L = E ;$

➤  $S \rightarrow \text{if } B \text{ then } S_1$   
     $\mid \text{if } B \text{ then } S_1 \text{ else } S_2$   
     $\mid \text{while } B \text{ do } S_1$

## ➤ 综合属性

➤ *S.nextlist*: 指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是按照运行顺序紧跟在S代码之后的指令的标号

**$S \rightarrow \text{if } B \text{ then } S_1$**

**$S \rightarrow \text{if } B \text{ then } M S_1$**

**{**

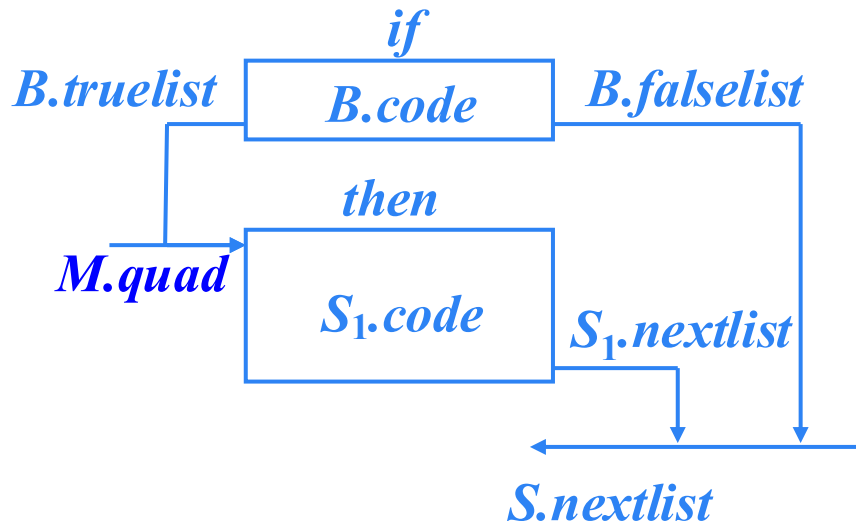
*$S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist});$*

*$\text{backpatch}(B.\text{truelist}, M.\text{quad});$*

**}**

**$M \rightarrow \varepsilon$**

**$\{ M.\text{quad} = \text{nextquad} ; \}$**



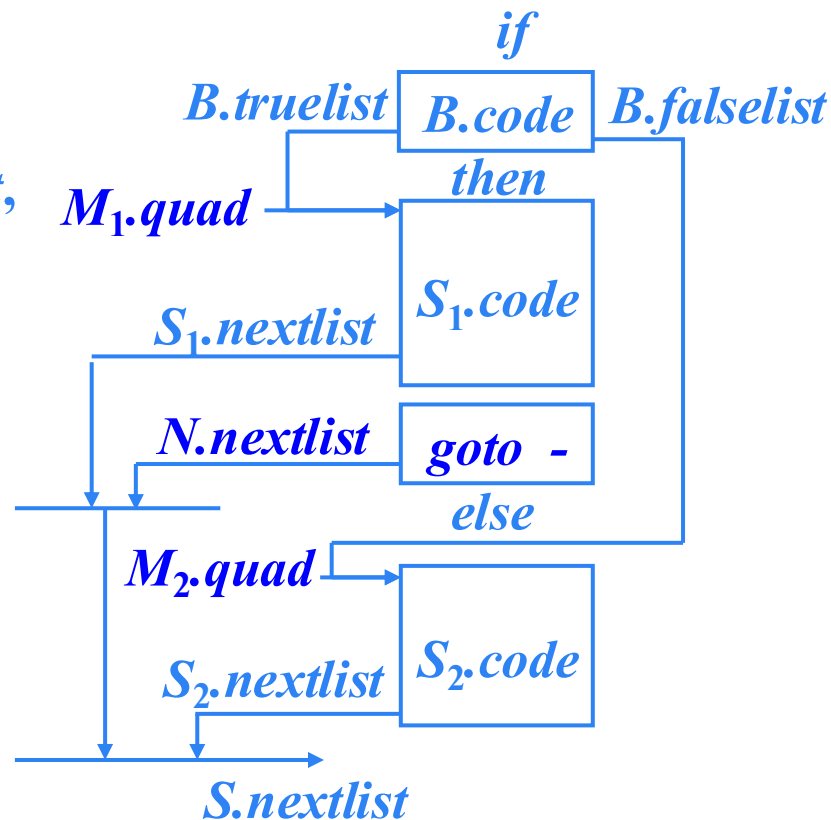
$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$

$S \rightarrow \text{if } B \text{ then } M_1 S_1 N \text{ else } M_2 S_2$

```
{
    S.nextlist = merge( merge(S1.nextlist,
                             N.nextlist), S2.nextlist );
    backpatch(B.truelist, M1.quad);
    backpatch(B.falselist, M2.quad);
}
```

$N \rightarrow \epsilon$

```
{
    N.nextlist = makelist(nextquad);
    gen('goto _');
}
```



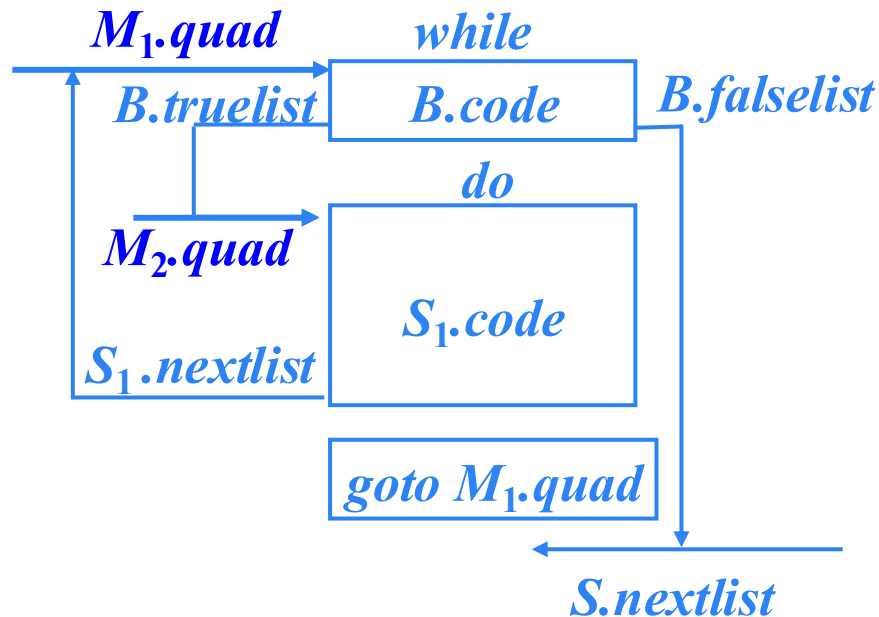
**$S \rightarrow \text{while } B \text{ do } S_1$**

$S \rightarrow \text{while } M_1 B \text{ do } M_2 S_1$

```
{  
   $S.\text{nextlist} = B.\text{falselist};$   
   $\text{backpatch}(S_1.\text{nextlist}, M_1.\text{quad});$   
   $\text{backpatch}(B.\text{truelist}, M_2.\text{quad});$   
   $\text{gen}(\text{'goto' } M_1.\text{quad});$   
}
```

$M \rightarrow \varepsilon$

```
{  $M.\text{quad} = \text{nextquad};$  }
```



$S \rightarrow S_1 S_2$

$S \rightarrow S_1 M S_2$

{

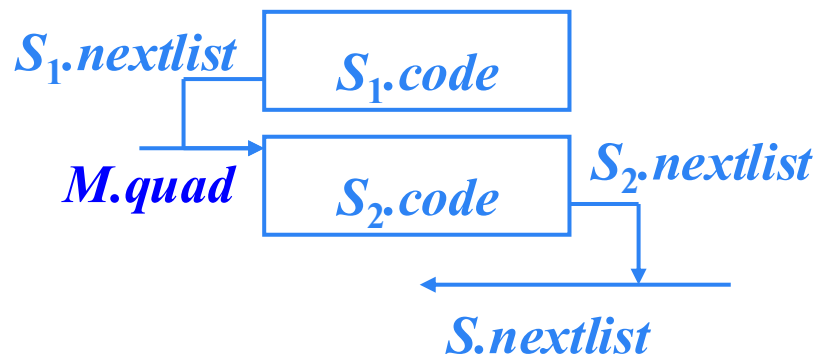
$S.nextlist = S_2.nextlist ;$

$backpatch( S_1.nextlist, M.quad );$

}

$M \rightarrow \varepsilon$

{  $M.quad = nextquad ;$  }






$$S \rightarrow \text{id} = E ; \mid L = E ;$$
$$S \rightarrow \text{id} = E ; \mid L = E ; \{ \textcolor{blue}{S.nextlist = null; } \}$$

# 例

*while*  $a < b$  *do*

*if*  $c < 5$  *then*

*while*  $x > y$  *do*  $z = x + 1$ ;

*else*

$x = y$ ;



*while a < b do if c < 5 then while x > y do z = x + 1; else x = y;*

100: *if a < b goto 102*

101: *goto \_*

102: *if c < 5 goto 104*

103: *goto 110*

104: *if x > y goto 106*

105: *goto 100*

106:  $t_1 = x + 1$

107:  $z = t_1$

108: *goto 104*

109: *goto 100*

110:  $x = y$

111: *goto 100*

112:

100: (*j* <, *a* , *b* , 102 )

101: (*j* , - , - , - )

102: (*j* <, *c* , 5 , 104 )

103: (*j* , - , - , 110 )

104: (*j* >, *x* , *y* , 106 )

105: (*j* , - , - , 100 )

106: ( + , *x* , 1 ,  $t_1$  )

107: ( = ,  $t_1$  , - ,  $z$  )

108: (*j* , - , - , 104 )

109: (*j* , - , - , 100 )

110: ( = , *y* , - , *x* )

111: (*j* , - , - , 100 )

112:

## 练习3

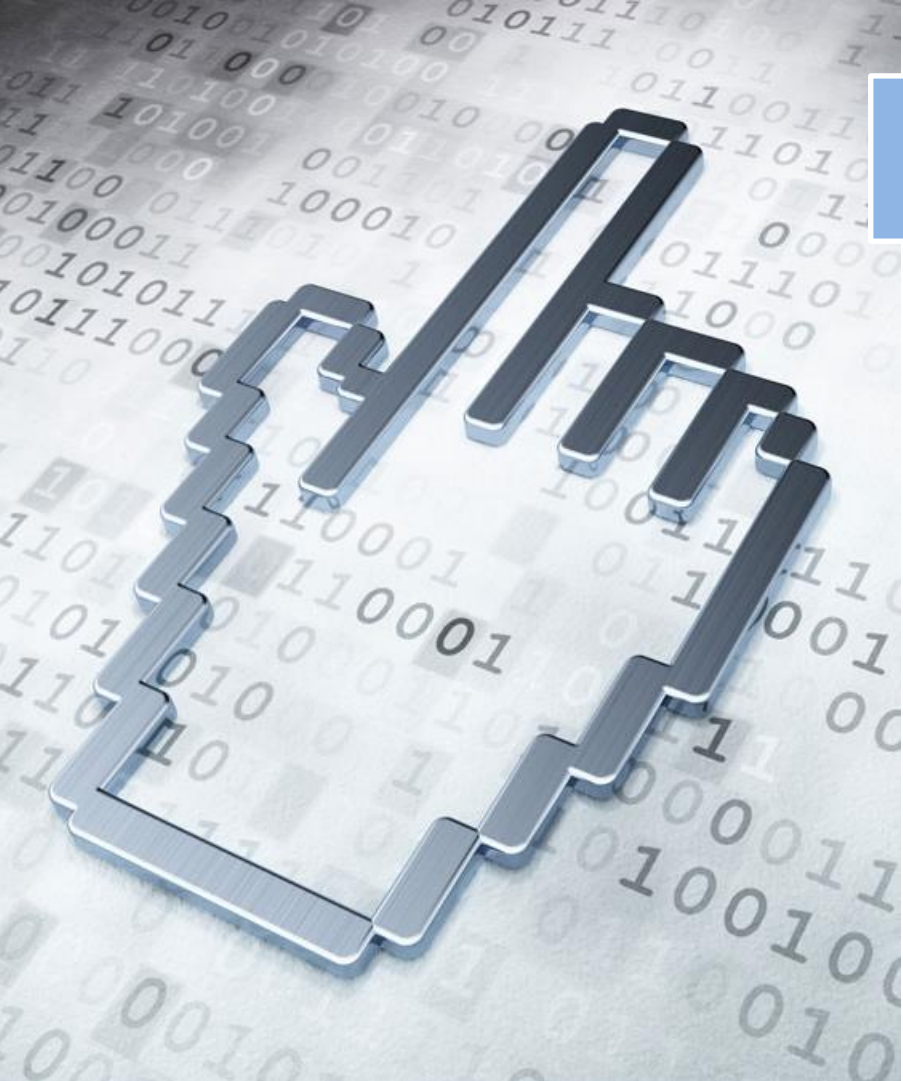
- 试将下面的语句翻译成三地址码和对应的四元式序列（指令标号从100开始，跳转指令目标标号是数字代表的序号）。不必考虑避免生成冗余的goto语句。

(1) while  $a < c$  and  $b < d$  do

    if  $a = 1$  then  $c = c + 1$

    else while  $a \leq d$  do

$a = a + 2$



# 本章内容

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 类型检查

6.4 控制语句的翻译

6.5 回填

**6.6 switch语句的翻译**

6.7 过程调用语句的翻译

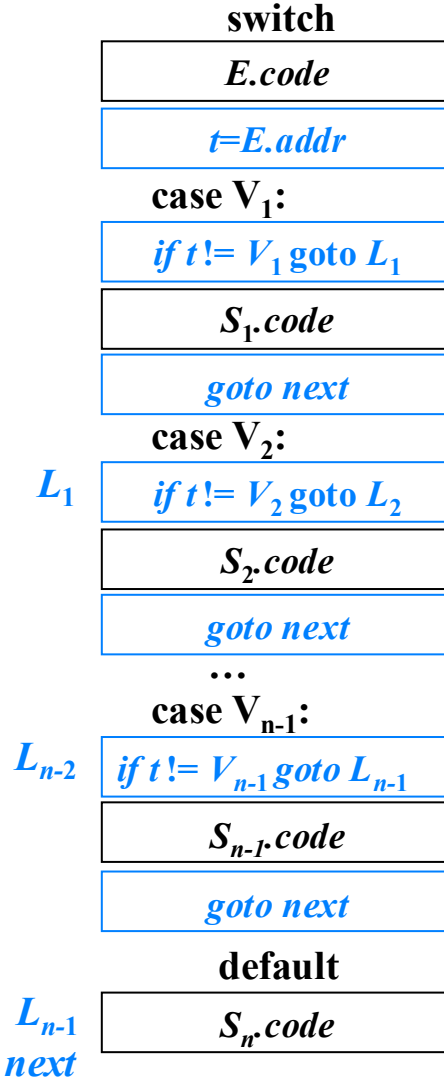
## 6.6 *switch*语句的翻译

### ➤ *switch*语句语法

```
switch E
begin
    case  $V_1$ :  $S_1$ 
    case  $V_2$ :  $S_2$ 
    ...
    case  $V_{n-1}$ :  $S_{n-1}$ 
    default:  $S_n$ 
end
```

### ➤ *switch*语句文法

```
 $S \rightarrow \text{switch } E \text{ begin } Clist \text{ end}$ 
 $Clist \rightarrow \text{case } V: S \ Clist$ 
 $Clist \rightarrow \text{default: } S$ 
```

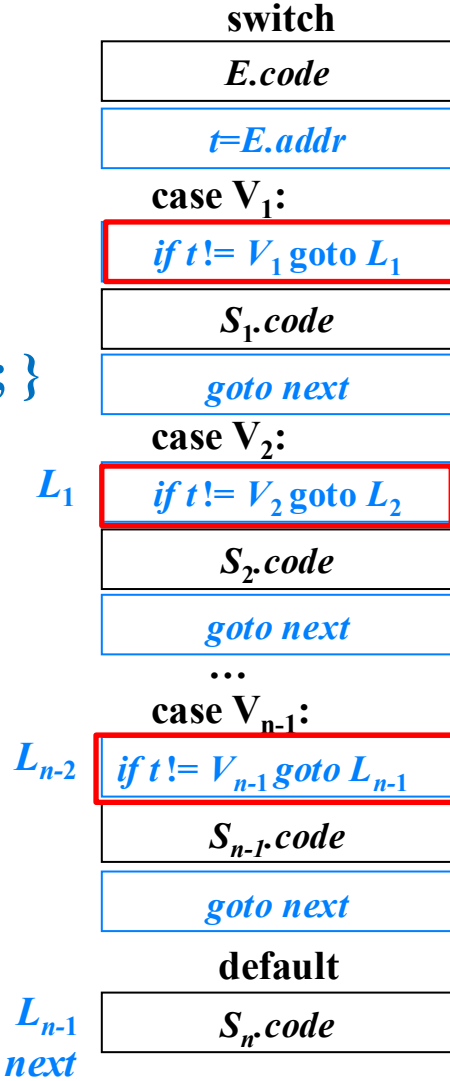


## 6.6 *switch*语句的翻译

$S \rightarrow \text{switch } E \{ t = \text{newtemp}();$   
      $\text{gen}(t \text{ '=' } E.\text{addr});$   
      $\text{next} = \text{newlabel}(); l = \text{newlabel}(); \}$   
 begin *Clist* end

*Clist*  $\rightarrow \text{case } V: \{ \text{label}(l); l = \text{newlabel}();$   
      $\text{gen}(\text{'if' } t \text{ '!=' } V \text{ 'goto' } l); \}$   
      $S \{ \text{gen}(\text{'goto' } \text{next}); \}$   
     *Clist*

*Clist*  $\rightarrow \text{default: } \{ \text{label}(l); \}$   
      $S \{ \text{label}(\text{next}); \}$





# *switch*语句的另一种翻译

switch  $E$

begin

case  $V_1$ :  $S_1$

case  $V_2$ :  $S_2$

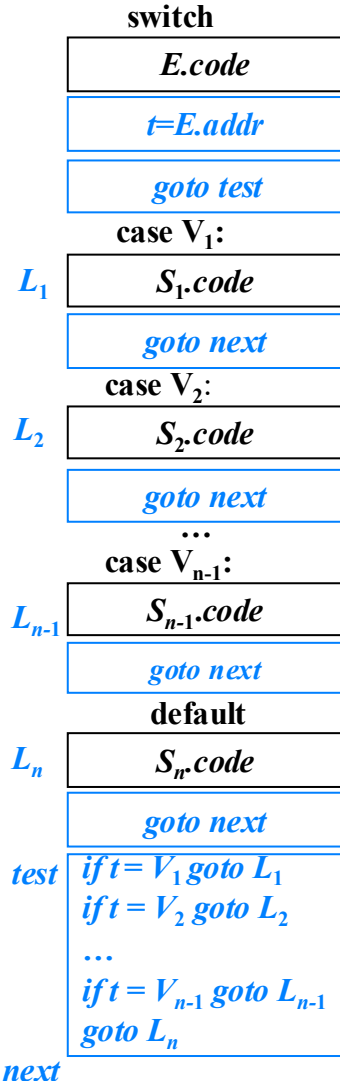
...

case  $V_{n-1}$ :  $S_{n-1}$

default:  $S_n$

end

**优势：**根据分支的个数以及这些值是否在一个较小的范围内，这种条件跳转指令序列可以被翻译成最高效的 $n$ 路分支(散列表)



# switch语句的另一种翻译

$S \rightarrow \text{switch } E \{ t = \text{newtemp}(); \text{gen}(t '=' E.addr);$   
 $\text{test} = \text{newlabel}(); \text{next} = \text{newlabel}();$   
 $\text{gen}(\text{'goto' test}); \}$

begin Clist end

$Clist \rightarrow \text{case } V: \{ l = \text{newlabel}(); \text{label}(l); q(V, l); \}$

$S \{ \text{gen}(\text{'goto' next}); \} Clist$

$Clist \rightarrow \text{default: } \{ l = \text{newlabel}(); \text{label}(l); \}$

$S \{ \text{gen}(\text{'goto' next}); \text{label}(\text{test});$

*for 队列 q 中的每对 (V<sub>i</sub>, l<sub>i</sub>) do*

*{*

*gen('if' t '=' V<sub>i</sub> 'goto' l<sub>i</sub>);*

*}*

$\text{gen}(\text{'goto' } l); // \text{ 跳转到 default } S$

$\text{label}(\text{next}); \}$

值-标号对加入队列q

switch

$E.code$

$t = E.addr$

$\text{goto test}$

case V<sub>1</sub>:

$S_1.code$

$\text{goto next}$

case V<sub>2</sub>:

$S_2.code$

$\text{goto next}$

...

case V<sub>n-1</sub>:

$S_{n-1}.code$

$\text{goto next}$

default

$S_n.code$

$\text{goto next}$

*test if t = V<sub>1</sub> goto L<sub>1</sub>*

*if t = V<sub>2</sub> goto L<sub>2</sub>*

...

*if t = V<sub>n-1</sub> goto L<sub>n-1</sub>*

*goto L<sub>n</sub>*

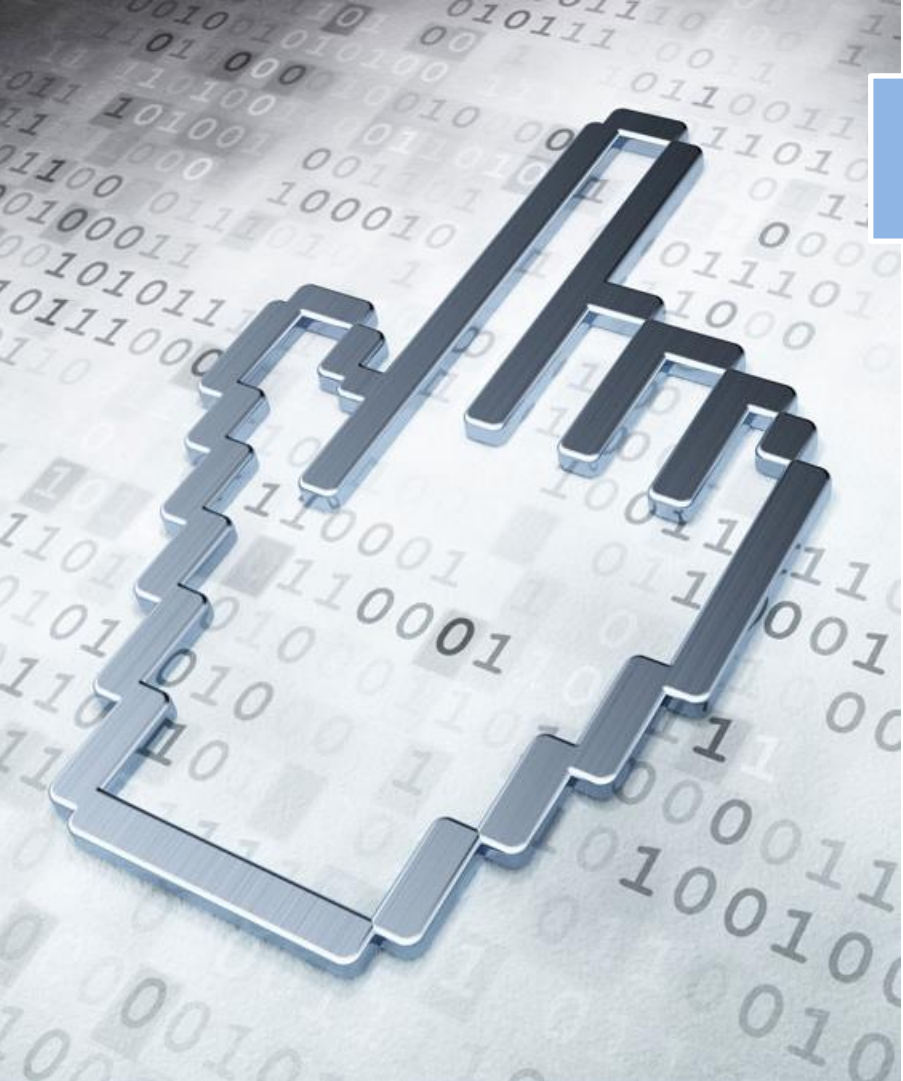
*next*

## 增加一种 *case* 指令

```
test :  if  $t = V_1$  goto  $L_1$   
        if  $t = V_2$  goto  $L_2$   
        ...  
        if  $t = V_{n-1}$  goto  $L_{n-1}$   
        goto  $L_n$   
next :
```

```
test :  case  $t$   $V_1$   $L_1$   
        case  $t$   $V_2$   $L_2$   
        ...  
        case  $t$   $V_{n-1}$   $L_{n-1}$   
        case  $t$   $t$   $L_n$   
next :
```

指令 *case*  $t$   $V_i$   $L_i$  和 *if*  $t = V_i$  goto  $L_i$  的含义相同，  
但是 *case* 指令更加容易被最终的代码生成器探测到，  
从而对这些指令进行特殊处理



# 本章内容

6.1 声明语句的翻译

6.2 赋值语句的翻译

6.3 类型检查

6.4 控制语句的翻译

6.5 回填

6.6 switch语句的翻译

**6.7 过程调用语句的翻译**

## 6.7 过程调用的翻译

### ➤ 文法

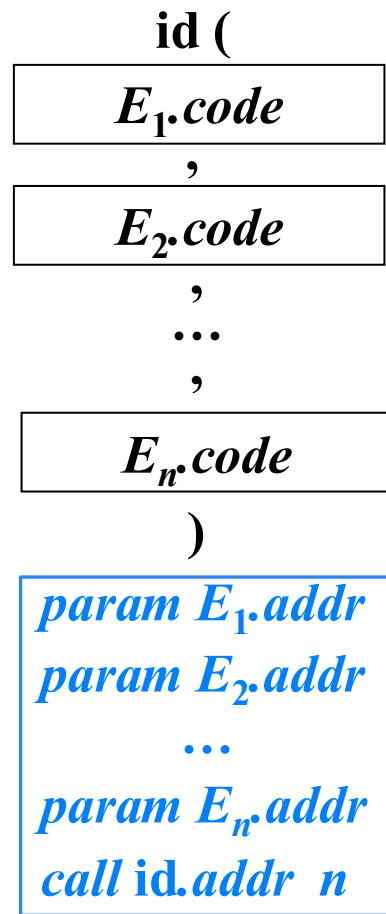
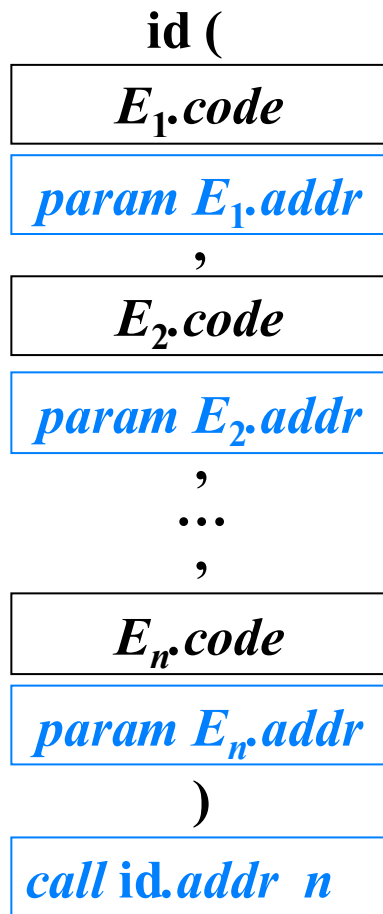
➤  $S \rightarrow \text{call id } (Elist)$

$Elist \rightarrow Elist, E$

$Elist \rightarrow E$

# 过程调用语句的代码结构

$\text{id}(E_1, E_2, \dots, E_n)$



# 过程调用语句的代码结构

$\text{id}(E_1, E_2, \dots, E_n)$

$\text{id} ($

$E_1.\text{code}$

,

$E_2.\text{code}$

,

$\dots$

,

$E_n.\text{code}$

)

$\left\{ \begin{array}{l} \text{param } E_1.\text{addr} \\ \text{param } E_2.\text{addr} \\ \dots \\ \text{param } E_n.\text{addr} \\ \text{call id.addr } n \end{array} \right.$

需要一个队列 $q$ 存放 $E_1.\text{addr}$ 、 $E_2.\text{addr}$ 、 $\dots$ 、 $E_n.\text{addr}$ ，以生成

# 过程调用语句的SDD

➤  $S \rightarrow \text{call id} ( Elist )$

```
{
     $n=0$ ;
    for  $q$  中的每个  $t$  do
    {
         $\text{gen}('param' t)$ ;
         $n = n+1$ ;
    }
     $\text{gen}('call' \text{id.addr}, n)$ ;
}
```

➤  $Elist \rightarrow E$

```
{
    将  $q$  初始化为只包含  $E.addr$ ;
}
```

➤  $Elist \rightarrow Elist_1, E$

```
{
    将  $E.addr$  添加到  $q$  的队尾;
}
```

```
id (
     $E_1.code$ 
    ,
     $E_2.code$ 
    ,
    ...
    ,
     $E_n.code$ 
)
```

```
 $param E_1.addr$ 
 $param E_2.addr$ 
...
 $param E_n.addr$ 
 $call \text{id.addr } n$ 
```



例：翻译以下语句  $f(b * c - 1, x + y, x, y)$

$$t_1 = b * c$$

$$t_2 = t_1 - 1$$

$$t_3 = x + y$$

*param*  $t_2$

*param*  $t_3$

*param*  $x$

*param*  $y$

*call*  $f, 4$

## 实验二指导

- 语义检查的核心任务是收集各种语法成分的类型信息，并进行类型匹配检查。
  - 类型的内部表示：P64 3.2.4 类型表示
    - 基本数据类型、数组和结构体
  - 类型的计算：综合属性或继承属性
  - 类型等价性：名字等价或结构等价
- 声明语句翻译的核心任务是收集标识符的类型信息并存入符号表
  - 不同种别的标识符可以建立不同的符号表
  - 符号表的数据结构 P60 3.2.2 符号表

## 实验二指导

- 语义检查的总体思想：类似递归的预测分析
  - 先构建语法分析树
  - 自顶向下深度优先遍历语法分析树，根据内部节点，分别调用对应的非终结符处理函数，按照对应的产生式进行解析，同时计算需要的属性值，或者进行类型检查。
  - 为非终结符编写解析函数
    - 函数计算并返回综合属性：如EXP的数据类型
    - 将继承属性作为函数参数：如声明语句中，ID的类型由父辈节点的左部符号specifier确定。

# 语义分析中的错误检测

- 变量或函数未经声明就使用（表达式/函数调用语句翻译）

```
EXP → ID  
EXP → ID LP Args RP  
EXP → ID LP RP
```

```
{
```

```
    //在符号表中查找ID，若未找到，则未声明
```

```
}
```

# 语义分析中的错误检测

- 变量或函数**未经声明就使用** (表达式/函数调用语句翻译)
- 变量或函数名**重复声明** (变量/函数声明语句翻译)

**VarDec  $\rightarrow$  ID**

**FunDec  $\rightarrow$  ID LR VarList RP**  
**| ID LP RP**

{

//在符号表中查找ID, 若找到了, 则为重复声明

}

# 语义分析中的错误检测

- 变量或函数**未经声明就使用** (表达式/函数调用语句翻译)
- 变量或函数名**重复声明** (变量/函数声明语句翻译)
- **赋值号**两边的表达式类型不匹配

**Dec** → **VarDec ASSIGNOP Exp**

**Exp** → **Exp ASSIGNOP Exp**

{

// 收集 ASSIGNOP 两侧的非终结符的数据类型，并进行类型比较

}

# 语义分析中的错误检测

- 变量或函数**未经声明就使用** (表达式/函数调用语句翻译)
- 变量或函数名**重复声明** (变量/函数声明语句翻译)
- **赋值号**两边的表达式类型不匹配
- 赋值号左边出现一个只有右值的表达式

```
Exp → Exp ASSIGNOP Exp
```

```
{
```

```
    //根据左值的可能形式, 使用启发性规则对EXP的子  
    结点进行判断
```

```
}
```

# 语义分析中的错误检测

- 变量或函数**未经声明就使用** (表达式/函数调用语句翻译)
- 变量或函数名**重复声明** (变量/函数声明语句翻译)
- **赋值号**两边的表达式类型不匹配
- 赋值号左边出现一个只有右值的表达式
- 操作数**类型不匹配**或操作数类型与操作符不匹配

**Stmt** → IF LP Exp RP Stmt | IF LP Exp RP Stmt ELSE Stmt

**Stmt** → WHILE LP Exp RP Stmt

**Exp** → NOT Exp | Exp AND Exp | Exp OR Exp | MINUS Exp | Exp RELOP Exp

**Exp** → Exp PLUS Exp | Exp MINUS Exp | Exp STAR Exp | Exp DIV Exp



# 语义分析中的错误检测

- 变量或函数**未经声明就使用**（表达式/函数调用语句翻译）
- 变量或函数名**重复声明**（变量/函数声明语句翻译）
- **赋值号**两边的表达式类型不匹配
- 赋值号左边出现一个只有右值的表达式
- 操作数**类型不匹配**或操作数类型与操作符不匹配
- **return**语句的返回类型与函数定义的返回类型不匹配

```
ExtDef → Specifier FunDec { CompSt.type = Specifier.type } CompSt
CompSt → LC DefList { StmtList.type = CompSt.type } StmtList RC
StmtList → {Stmt.type = StmtList.type} Stmt
Stmt → RETURN Exp SEMI { if Exp.type = Stmt.type ? }
```

# 语义分析中的错误检测

- 变量或函数**未经声明就使用**（表达式/函数调用语句翻译）
- 变量或函数名**重复声明**（变量/函数声明语句翻译）
- **赋值号**两边的表达式类型不匹配
- 赋值号左边出现一个只有右值的表达式
- 操作数**类型不匹配**或操作数类型与操作符不匹配
- **return**语句的返回类型与函数定义的返回类型不匹配
- 函数调用时形参的数目或类型不匹配

$\text{Exp} \rightarrow \text{ID LP Args RP}$

$\text{Exp} \rightarrow \text{ID LP RP}$

{ //在函数表中查找ID可知函数的形参数目和类型，解析Args可知实参数目和类型，然后进行比较。 }

# 语义分析中的错误检测

- 变量或函数**未经声明就使用**（表达式/函数调用语句翻译）
- 变量或函数名**重复声明**（变量/函数声明语句翻译）
- **赋值号**两边的表达式类型不匹配
- 赋值号左边出现一个只有右值的表达式
- 操作数**类型不匹配**或操作数类型与操作符不匹配
- **return**语句的返回类型与函数定义的返回类型不匹配
- 函数调用时形参的数目或类型不匹配
- 对非数组型变量使用数组访问操作符，数组访问[]中出现非整数

Exp  $\rightarrow$  Exp LB Exp RB { if Exp.type = 整型 ? }

# 语义分析中的错误检测

- 变量或函数**未经声明就使用**（表达式/函数调用语句翻译）
- 变量或函数名**重复声明**（变量/函数声明语句翻译）
- **赋值号**两边的表达式类型不匹配
- 赋值号左边出现一个只有右值的表达式
- 操作数**类型不匹配**或操作数类型与操作符不匹配
- **return**语句的返回类型与函数定义的返回类型不匹配
- 函数调用时形参的数目或类型不匹配
- 对非数组型变量使用数组访问操作符，数组访问[]中出现非整数
- 对普通变量使用函数调用操作符

Exp  $\rightarrow$  ID LP RP | ID LP Args RP  
{ //在符号表中查找ID，是否函数? }

# 语义分析中的错误检测

## ➤ 结构体类错误

### ➤ 对非结构体型变量使用“.”操作符

$\text{Exp} \rightarrow \text{Exp DOT ID}$

{ //解析Exp的子结点, 收集  
Exp.type, 是否结构体? }

### ➤ 结构体域名重复定义, 或在定义时对域进行了初始化

$\text{VarDec} \rightarrow \text{ID}$  { //若是声明结构体中的域, 要检查是否重复定义, 需要记录  
是哪种声明使用了这个产生式, 函数声明/定义? 结构体声明? 变量声明? }

$\text{Dec} \rightarrow \text{VarDec ASSIGNOP Exp}$  { //若是声明结构体的域, 则不合法 }

### ➤ 结构体的名字与前面定义过的结构体或变量的名字重复

$\text{OptTag} \rightarrow \text{ID}$  { //在符号表中查找ID, 若找到则重名 }

### ➤ 使用未定义过的结构体来定义变量

$\text{Tag} \rightarrow \text{ID}$  { //在符号表中查找ID, 若未找到或种别不是结构体, 则未定义 }

# 本章小结

- 声明语句的翻译
  - 变量声明的翻译
  - 数组声明的翻译
- 赋值语句的翻译
  - 简单赋值语句的翻译
  - 数组引用的翻译
- 控制语句的翻译
- 回填
- switch语句的翻译
- 过程调用语句的翻译

# 课程主要内容

1. 绪论 (2学时)
2. 语言及其文法 (2学时)
3. 词法分析 (3学时)
4. 语法分析 (9学时)
5. 语法制导翻译 (6学时)
6. 中间代码生成 (7学时)
7. 运行时的存贮组织 (3学时)
8. 代码优化 (6学时)
9. 代码生成 (2学时)



# 结束

